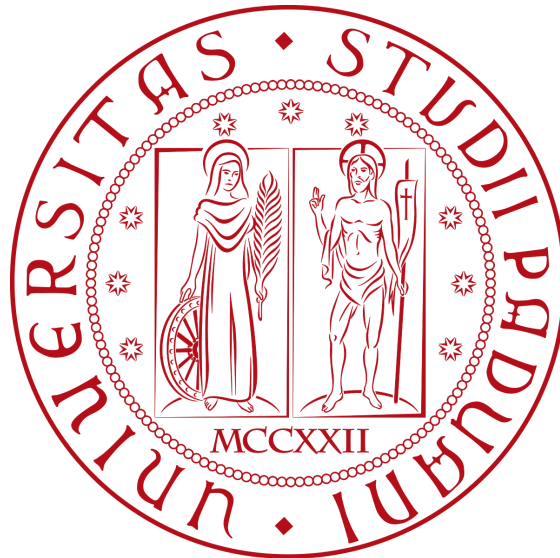




ELEMENTI DI INGEGNERIA DEL SOFTWARE

1° APPELLO A.A. 2026

Corso di Laurea in Ingegneria Informatica L-8

**ZORZI ALESSANDRO
N° MATRICOLA: 2137471**

11.06.26

INDICE

Note informative

ListAdapterConstructorTest

ListAdapterBasicOperationsTest

- ListAdapter.size()
- ListAdapter.isEmpty()
- ListAdapter.add(Object o)
- ListAdapter.remove(Object o)
- ListAdapter.clear()
- ListAdapter.contains(Object o)
- ListAdapter.equals(Object o)
- ListAdapter.hashCode()

ListAdapterCollectionTest

- ListAdapter.containsAll(HCollection c)
- ListAdapter.addAll(HCollection c)
- ListAdapter.addAll(int index, HCollection c)
- ListAdapter.removeAll(HCollection c)
- ListAdapter.retainAll(HCollection c)

ListAdapterToArrayTest

- toArray()
- toArray(Object[] a)

ListAdapterIndexedOperationsTest

- ListAdapter.get(int index)
- ListAdapter.set(int index, Object element)
- ListAdapter.add(int index, Object element)
- ListAdapter.remove(int index)

ListAdapterSearchTest

- ListAdapter.indexOf(Object o)
- ListAdapter.lastIndexOf(Object o)

ListAdapterIteratorTest

- Costruttore
- ListAdapter.iterator()
- IteratorAdapter.hasNext()
- IteratorAdapter.next()
- IteratorAdapter.remove()

ListAdapterListIteratorTest

- ListAdapter.listIterator()
- ListAdapter.listIterator(index)
- ListIteratorAdapter.hasPrevious()
- ListIteratorAdapter.previous()
- ListIteratorAdapter.nextIndex() e ListIteratorAdapter.previousIndex()
- ListIteratorAdapter.set(Object o)
- ListIteratorAdapter.add(Object o)
- ListIteratorAdapter.remove()

ListAdapterSubListTest

- ListAdapter.subList(int fromIndex, int toIndex)
- SubList.set(int index, Object element)
- SubList.add(int index, Object element)
- SubList.add(Object o)
- SubList.remove(int index)
- SubList.clear()
- SubList.contains(Object o), indexOf(Object o), lastIndexOf(Object o)
- SubList.containsAll(HCollection c), addAll(HCollection c), removeAll(HCollection c), retainAll(HCollection c)
- SubList.subList(int fromIndex, int toIndex)
- SubList.toArray()
- SubList.toArray(Object[] a)
- SubList.iterator() e listIterator(int index)
- SubList.equals(Object o) e hashCode()
- Eccezioni su creazione subList e su metodi indicizzati

Note informative

- **Versione JUnit**

Tutti i test sono stati implementati utilizzando il framework JUnit 4.13.

- **Test**

In ciascun test in cui viene verificato il comportamento di uno specifico metodo, gli eventuali altri metodi utilizzati per costruire lo stato iniziale della lista o per controllare il risultato finale vengono considerati corretti, in quanto oggetto di test specifici all'interno della documentazione.

Ad esempio, per verificare il corretto funzionamento di `remove(Object o)` può essere necessario usare `add(Object o)` per popolare la lista, oppure `size()`, `get(int index)` e `contains(Object o)` per controllare lo stato finale. Tali metodi non sono quindi il vero oggetto del test in quel momento, ma vengono usati come strumenti di supporto.

Questa scelta è necessaria perché molti metodi della classe `ListAdapter` dipendono tra loro durante la fase di testing: non sarebbe possibile verificare ogni metodo in modo completamente isolato senza utilizzare almeno alcune operazioni già presenti nella lista. Per questo motivo si è deciso di dare per assodato il corretto funzionamento dei metodi di supporto, rimandando la loro verifica ai test case dedicati.

- **@Before**

Nei metodi `Before` sono state inizializzate solamente liste vuote, evitando di creare direttamente liste già popolate con elementi predefiniti (fatta eccezione per il test case `ListAdapterSubListTest`).

In alcuni test case sarebbe stato possibile predisporre nel `Before` liste già pronte, ad esempio una lista contenente "A", "B" e "C", così da evitare di ripetere più volte le stesse chiamate ad `add(Object o)` all'interno dei singoli test. Tuttavia, per scelta progettuale, si è preferito costruire esplicitamente lo stato iniziale della lista dentro ogni test method.

Questa soluzione rende ogni test più chiaro, autonomo e facilmente leggibile, perché permette di vedere direttamente quali elementi vengono inseriti prima dell'operazione da verificare e quale situazione iniziale viene effettivamente testata. Inoltre, evita che il comportamento di un test dipenda in modo poco visibile da una configurazione generale definita nel `before`.

La ripetizione di alcune istruzioni di inserimento è stata quindi accettata per aumentare la comprensibilità e la precisione della documentazione, rendendo evidente, per ogni singolo test, il rapporto tra stato iniziale, operazione eseguita e risultato atteso.

- **toArray(Object[] a)**

Relativamente al metodo `toArray(Object[] a)`, non è stato implementato il controllo completo del runtime type dell'array passato come parametro.

Secondo la documentazione della `Collection` di Java, il metodo dovrebbe restituire un array dello stesso tipo runtime dell'array ricevuto come parametro e, nel caso in cui il tipo dell'array non sia compatibile con gli elementi della lista, dovrebbe poter generare `ArrayStoreException`.

Nel progetto, tuttavia, l'implementazione è stata realizzata utilizzando solamente le funzionalità disponibili nell'ambiente CLDC 1.1, che non fornisce (secondo il progettista) strumenti sufficienti per creare dinamicamente un nuovo array mantenendo il tipo runtime specifico dell'array passato.

Per questo motivo, nel caso in cui l'array fornito non abbia dimensione sufficiente, il metodo crea un nuovo array di tipo `Object[]` e non un array dello stesso tipo runtime del parametro. La documentazione dei test tiene conto di questa scelta implementativa.

- **ListAdapterListIteratorTest**

Relativamente al test case `ListAdapterListIteratorTest`, i metodi ereditati da `HIterator`, cioè `hasNext()`, `next()` e `remove()`, non sono stati tutti ritestati in modo completo, perché il loro comportamento generale è già stato verificato nel test case dedicato a `IteratorAdapter`.

Per questo motivo, nel test case relativo al list iterator si è scelto di concentrare l'attenzione sui metodi specifici aggiunti da `HListIterator`, come `hasPrevious()`, `previous()`, `nextIndex()`, `previousIndex()`, `set(Object o)` e `add(Object o)`.

Tra i metodi ereditati è stato ripreso soprattutto `remove()`, perché nel caso del list iterator presenta un comportamento più ampio e delicato rispetto all'iterator semplice: l'elemento da rimuovere può infatti essere l'ultimo restituito sia da `next()` sia da `previous()`.

- **Null**

La documentazione di `List` prevede, per alcuni metodi, la possibilità che determinate implementazioni non accettino elementi null. Nel caso di `ListAdapter`, tuttavia, si è scelto di considerare tale elemento come accettabile.

Per questo motivo, sono stati eseguiti dei test method specifici per verificare la corretta gestione di null nei diversi metodi della lista, che non provochi eccezioni, salvo nei casi in cui l'eccezione sia dovuta ad altre condizioni non valide, come indici fuori limite o parametri `collection/array` nulli.

- **ListAdapterSubListTest**

Relativamente al test case `ListAdapterSubListTest`, non sono stati riscritti test completi per ogni singolo metodo già verificato in modo esteso sulla classe `ListAdapter`.

- **IllegalStateException**

Poiché l'ambiente CLDC 1.1 non include esplicitamente `java.lang.IllegalStateException`, nei casi in cui la documentazione standard prevede `IllegalStateException` è stata utilizzata `RuntimeException` come eccezione runtime sostitutiva.

- **Documentazione**

Relativamente alla generazione della documentazione Javadoc, come richiesto dalla consegna essa è stata prodotta per il codice del progetto. Alcuni warning relativi all'assenza di commenti sono dovuti al fatto che non tutti gli elementi sono stati documentati tramite Javadoc: per le interfacce, utilizzate principalmente per rispettare la struttura richiesta ed evitare ambiguità di omonimia con le classi standard, è stata ritenuta sufficiente la chiarezza delle firme dei metodi; per le classi di test, invece, si è scelto di fornire una documentazione esterna in formato PDF, più estesa e leggibile rispetto ai commenti Javadoc.

- **Utilizzo intelligenza artificiale**

Per la realizzazione del progetto e della relativa documentazione è stato utilizzato anche un LLM, nello specifico ChatGPT basato su GPT-5.5 Thinking, come supporto per rendere più completa, ordinata e leggibile la documentazione prodotta.

L'utilizzo dello strumento di IA ha avuto funzione esclusivamente di supporto allo sviluppo e alla revisione del lavoro. In particolare, è stato impiegato per:

- attività di analisi generale del codice;
- individuazione di possibili casi limite da testare;
- revisione ortografica;
- miglioramento della chiarezza espositiva del testo.

ListAdapterConstructorTest

Summary

Il test case ListAdapterConstructorTest verifica il corretto stato iniziale di una nuova istanza di ListAdapter.

L'obiettivo è controllare che, subito dopo la costruzione, la lista sia vuota, abbia dimensione zero, produca un array vuoto, abbia un hashCode pari a 1 e che le operazioni non valide su lista vuota generino le eccezioni attese.

Test Case Design

Ogni test crea una nuova istanza di ListAdapter, inizializzata prima dell'esecuzione tramite il metodo setUp().

I test verificano sia il comportamento normale del costruttore, sia alcuni casi limite legati all'uso di una lista vuota.

Test

- public void testConstructorSizeZero()

Summary: Verifica che una lista appena costruita abbia dimensione pari a zero.

Test Method Design: Il test crea una nuova lista e richiama il metodo size().

Test Description: Viene controllato che il valore restituito da size() sia 0.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: size() == 0.

- public void testConstructorIsEmpty()

Summary: Verifica che una lista appena costruita risulti vuota.

Test Method Design: Il test crea una nuova lista e richiama isEmpty()

Test Description: Viene controllato che isEmpty() indichi correttamente l'assenza di elementi nella lista

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane invariata e vuota.

Expected result: isEmpty() == true

- public void testConstructorToArray()

Summary: Verifica che una lista appena costruita risulti vuota.

Test Method Design: Il test crea una nuova lista e richiama toArray().

Test Description: Viene controllato che l'array creato con il metodo toArray() indichi correttamente l'assenza di elementi nella lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: array.length sull'array generato da toArray() == 0.

- **public void testConstructorHashCode()**

Summary: Verifica che una lista appena costruita risulti vuota.

Test Method Design: Il test crea una nuova lista e richiama hashCode().

Test Description: Viene controllato che il valore restituito da hashCode() indichi correttamente l'assenza di elementi nella lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: hashCode() == 1.

- **public void testConstructorIteratorHasNextFalse()**

Summary: Verifica che una lista appena costruita risulti vuota.

Test Method Design: Il test crea una nuova lista ed un iteratore su di essa e richiama hasNext().

Test Description: Viene controllato che il valore restituito da hasNext() su un iteratore appena creato sulla lista indichi correttamente l'assenza di elementi nella lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: iterator().hasNext() == false.

- **public void testConstructorGet()**

Summary: Verifica che get(int Index) su una lista appena costruita generi un'eccezione.

Test Method Design: Il test crea una nuova lista e chiama il metodo get(0).

Test Description: Viene controllato che il valore restituito (o meglio, l'eccezione lanciata) da get(0) indichi correttamente l'assenza di elementi nella lista (è stato scelto l'indice 0 perchè se non c'è un elemento nella prima posizione, sicuramente non ce ne saranno in quelle successive).

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: get(0) lancia IndexOutOfBoundsException.

- **public void testConstructorSet()**

Summary: Verifica che set(0, Object) su una lista appena costruita generi un'eccezione.

Test Method Design: Il test crea una nuova lista e chiama il metodo set(0, "A").

Test Description: Viene controllato che il valore restituito (o meglio, l'eccezione lanciata) da set(0, "A") (al posto di "A" si poteva mettere qualsiasi altro object), indichi correttamente l'assenza di elementi nella lista (è stato scelto l'indice 0 perchè se non c'è un elemento nella prima posizione, sicuramente non ce ne saranno in quelle successive).

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: set(0, "A") lancia IndexOutOfBoundsException.

- **public void testConstructorRemove()**

Summary: Verifica che remove(0) su una lista appena costruita generi un'eccezione.

Test Method Design: Il test crea una nuova lista e chiama il metodo remove(0).

Test Description: Viene controllato che il valore restituito (o meglio, l'eccezione lanciata) da remove(0) indichi correttamente l'assenza di elementi nella lista (è stato scelto l'indice 0 perchè se non c'è un elemento nella prima posizione, sicuramente non ce ne saranno in quelle successive).

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: remove(0) lancia IndexOutOfBoundsException.

- **public void testConstructorNext()**

Summary: Verifica che next() su una lista appena costruita generi un'eccezione

Test Method Design: Il test crea una nuova lista e chiama il metodo, tramite un iteratore di default, il metodo next()

Test Description: Viene controllato che il valore restituito (o meglio, l'eccezione lanciata) da next() indichi correttamente l'assenza di elementi nella lista (è stato scelto l'indice 0 perchè se non c'è un elemento nella prima posizione, sicuramente non ce ne saranno in quelle successive).

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: iterator().next() lancia NoSuchElementException

ListAdapterBasicOperationsTest

Summary

Il test case ListAdapterBasicOperationsTest verifica il corretto funzionamento delle operazioni di base della classe ListAdapter.

L'obiettivo è controllare che i metodi fondamentali della lista gestiscano correttamente l'inserimento, la rimozione, lo svuotamento, la ricerca semplice, il confronto tra liste e il calcolo dell'hash code.

In particolare, vengono testati i metodi:

- size();
- isEmpty();
- add(Object o);
- remove(Object o);
- clear();
- contains(Object o);
- equals(Object o);
- hashCode().

I test considerano sia casi ordinari, sia casi limite, come lista vuota, elementi duplicati, elementi null, oggetti non presenti e confronto tra liste diverse.

Test Case Design

Ogni test crea o utilizza una nuova istanza di ListAdapter, inizializzata prima dell'esecuzione tramite il metodo setUp().

I test sono organizzati per metodo testato. Per ogni metodo vengono verificati sia il valore restituito, quando presente, sia gli effetti prodotti sullo stato interno della lista.

Le verifiche controllano quindi la dimensione della lista, il contenuto degli elementi, l'ordine di inserimento, la presenza o assenza di determinati oggetti e la coerenza tra equals() e hashCode().

Test

- **ListAdapter.size()**

- **public void testSizeOnEmptyList()**

Summary: Verifica che una lista appena creata abbia dimensione pari a zero.

Test Method Design: Il test crea una nuova lista e richiama il metodo size().

Test Description: Viene controllato che il valore restituito da size() indichi correttamente l'assenza di elementi nella lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: size() == 0.

- **public void testSizeAfterAdd()**

Summary: Verifica che la dimensione della lista aumenti dopo l'inserimento di elementi.

Test Method Design: Il test inserisce 2 elementi nella lista tramite add(Object o) e poi richiama size().

Test Description: Viene controllato che la lista abbia dimensione pari al numero di elementi inseriti.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B".

Expected result: size() == 2.

- **public void testSizeAfterInsertRemove()**

Summary: Verifica che la dimensione della lista diminuisca dopo la rimozione di un elemento precedentemente aggiunto.

Test Method Design: Il test inserisce due elementi, ne rimuove uno tramite remove(Object o) e poi richiama size().

Test Description: Viene controllato che la dimensione della lista venga aggiornata correttamente dopo la rimozione.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "B".

Expected result: size() == 1.

- **public void testSizeAfterInsertRemove1()**

Summary: Verifica che la dimensione torni a zero dopo aver fatto tante rimozioni quante sono stati gli inserimenti.

Test Method Design: Il test inserisce 2 elementi, li rimuove entrambi e poi chiama size().

Test Description: Viene controllato che la lista risulti vuota dopo la rimozione completa degli elementi.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota (tutti gli elementi inseriti sono stati rimossi).

Expected result: size() == 0

- **public void testSizeAfterClear()**

Summary: Verifica che clear() riporti la dimensione della lista a zero

Test Method Design: Il test inserisce 2 elementi, richiama clear() e poi controlla size().

Test Description: Viene controllato che lo svuotamento della lista azzeri correttamente il numero di elementi.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista è vuota.

Expected result: size() == 0

- **ListAdapter.isEmpty()**

- **public void testIsEmptyOnEmptyList()**

Summary: Verifica che una lista appena creata risulti vuota.

Test Method Design: Il test richiama isEmpty() su una nuova istanza di ListAdapter.

Test Description: Viene controllato che isEmpty() riconosca l'assenza di elementi.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: isEmpty() == true.

- **public void testIsEmptyAfterAdd()**

Summary: Verifica che una lista non risulti vuota dopo l'inserimento di un elemento.

Test Method Design: Il test inserisce un elemento nella lista e richiama isEmpty().

Test Description: Viene controllato che la presenza di almeno un elemento venga rilevata correttamente (il fatto che si controlli tramite l'inserimento di un singolo elemento è puramente per semplicità, non viene considerato come caso critico l'inserimento di più elementi in successione, per questa casistica).

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene un elemento.

Expected result: isEmpty() == false.

- **public void testIsEmptyAfterRemove()**

Summary: Verifica che una lista torni vuota dopo l'inserimento e la successiva rimozione dell'unico elemento.

Test Method Design: Il test inserisce un elemento, lo rimuove e richiama isEmpty().

Test Description: Viene controllato che la lista riconosca correttamente lo stato vuoto dopo una rimozione (anche in questo caso un singolo elemento per semplicità).

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista è vuota.

Expected result: isEmpty() == true

- **public void testIsEmptyAfterAddAddRemove()**

Summary: Verifica che una lista non risulti vuota se, dopo una rimozione, contiene ancora elementi.

Test Method Design: Il test inserisce due elementi, ne rimuove uno e richiama isEmpty().

Test Description: Vengono inseriti due elementi nella lista e successivamente viene rimossa una sola delle due occorrenze presenti. Il test serve a verificare che il metodo isEmpty() valuti lo stato effettivo della lista dopo una modifica parziale, senza considerarla vuota solo perché è stata eseguita un'operazione di rimozione.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "B".

Expected result: isEmpty() == false.

- **public void testIsEmptyAfterClear()**

Summary: Verifica che una lista risulti vuota dopo l'esecuzione di clear()

Test Method Design: Il test inserisce un elemento, richiama clear() e poi isEmpty().

Test Description: Il test costruisce una situazione in cui una lista non vuota viene svuotata tramite il metodo clear(). Dopo l'operazione di svuotamento viene richiamato isEmpty() per verificare che lo stato della lista sia stato aggiornato correttamente. Il test è utile perché controlla la coerenza tra una modifica strutturale della lista e il metodo che ne rileva la mancanza di elementi.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista è vuota.

Expected result: isEmpty() == true.

- **ListAdapter.add(Object o)**

- **public void testAddReturnsTrue()**

Summary: Verifica che il metodo `add(Object o)` restituisca `true` dopo l'inserimento di un elemento.

Test Method Design: Il test inserisce un elemento nella lista tramite `add()` e controlla il valore booleano restituito.

Test Description: Il metodo `add("A")` viene eseguito su una lista appena creata. Poiché l'operazione di inserimento in coda è supportata dall'adapter, il metodo deve indicare che la lista è stata modificata correttamente.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene l'elemento "A".

Expected result: `add("A") == true`.

- **public void testAddActuallyAddsElement()**

Summary: Verifica che il metodo `add(Object o)` non si limiti a restituire `true`, ma inserisca effettivamente l'elemento nella lista.

Test Method Design: Il test inserisce un elemento tramite `add()` e poi usa `get(0)` per controllare che l'elemento sia stato realmente memorizzato nella prima posizione della lista.

Test Description: Viene aggiunto l'elemento "A" a una lista inizialmente vuota. Dopo l'inserimento, il contenuto della posizione 0 deve corrispondere all'elemento appena inserito.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene l'elemento "A" in indice 0.

Expected result: `get(0) == "A"`.

- **public void testAddKeepsInsertionOrder()**

Summary: Verifica che più chiamate consecutive ad `add(Object o)` mantengano l'ordine di inserimento degli elementi.

Test Method Design: Il test inserisce tre elementi in sequenza e poi controlla, tramite `get()`, che siano presenti nella lista nello stesso ordine in cui sono stati aggiunti.

Test Description: Gli elementi "A", "B" e "C" vengono inseriti uno dopo l'altro. Poiché `ListAdapter` deve comportarsi come una lista, gli elementi devono essere accessibili rispettando l'ordine di inserimento.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene tre elementi nell'ordine di inserimento.

Expected result: `get(0) == "A"`, `get(1) == "B"`, `get(2) == "C"`.

- **public void testAddAllowsDuplicates()**

Summary: Verifica che la lista permetta l'inserimento di elementi duplicati.

Test Method Design: Il test inserisce due volte lo stesso oggetto e controlla sia la dimensione della lista sia la presenza delle due occorrenze.

Test Description: Gli elementi "A" e "A" vengono inseriti uno dopo l'altro. Verifico successivamente che entrambe le occorrenze siano presenti (la lista ammette duplicati)

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene due elementi, entrambi uguali a "A"

Expected result: `size() == 2; get(0) == get(1) == "A".`

- **public void testAddNullElement()**

Summary: Verifica che il metodo `add(Object o)` consenta l'inserimento di un elemento null, come un qualsiasi altro elemento.

Test Method Design: Il test inserisce null nella lista e poi verifica che la lista lo contenga correttamente.

Test Description: Viene chiamato `add(null)` su una lista vuota. Il test controlla che null venga gestito come elemento valido della lista e che sia rilevabile tramite `contains(null)`.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene un elemento null.

Expected result: `size() == 1, contains(null) == true.`

- **ListAdapter.remove(Object o)**

- **public void testRemoveExistingElement()**

Summary: Verifica che remove(Object o) restituisca true quando l'elemento da rimuovere è presente nella lista.

Test Method Design: Il test inserisce un elemento nella lista e successivamente richiama remove() sullo stesso elemento. Il valore restituito viene controllato con assertTrue().

Test Description: Dopo aver aggiunto "A", viene eseguita la rimozione dello stesso elemento. Poiché l'elemento è presente, l'operazione deve riuscire e il metodo deve segnalare che la lista è stata modificata.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: remove("A") == true

- **public void testRemoveExistingElement1()**

Summary: Verifica che la rimozione di un elemento presente abbia effetto reale sul contenuto della lista.

Test Method Design: Il test inserisce due elementi, rimuove il primo e poi controlla che l'elemento rimosso non sia più presente, che la dimensione sia aggiornata e che l'elemento residuo sia nella posizione corretta.

Test Description: La lista viene inizializzata con "A" e "B". Dopo remove("A"), "A" non deve più essere contenuto nella lista e "B" deve diventare il primo elemento.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene solo "B"

Expected result: contains("A") == false, size() == 1, get(0) == "B"

- **public void testRemoveNonExistingElement()**

Summary: Verifica che remove(Object o) restituisca false quando l'elemento richiesto non è presente nella lista.

Test Method Design: Il test crea una lista contenente un elemento e tenta di rimuovere un elemento diverso. Il risultato atteso è false.

Test Description: La lista contiene "A", ma viene richiesta la rimozione di "B". Poiché "B" non è contenuto nella lista, non deve avvenire alcuna modifica.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: remove("B") == false

- **public void testRemoveOnlyFirstOccurrence()**

Summary: Verifica che remove(Object o) elimini solo la prima occorrenza dell'elemento specificato.

Test Method Design: Il test inserisce elementi duplicati nella lista e richiama remove() una sola volta. Successivamente controlla che sia stata rimossa solo la prima occorrenza.

Test Description: La lista contiene "A", "B" e "A". Quando viene chiamato remove("A"), deve essere eliminata solo la prima "A", mentre la seconda deve rimanere nella lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene una sola occorrenza di "A"

Expected result: size() == 2, get(0) == "B", get(1) == "A".

- **public void testRemoveFromEmptyList()**

Summary: Verifica il comportamento di remove(Object o) quando viene chiamato su una lista vuota.

Test Method Design: Il test richiama direttamente remove("A") senza inserire prima alcun elemento.

Test Description: Poiché la lista è vuota, non esiste alcun elemento da rimuovere. Il metodo deve quindi terminare senza modifiche e restituire false.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: remove("A") == false.

- **public void testRemoveNonExistingDoesNotModifyList()**

Summary: Verifica che il tentativo di rimuovere un elemento non presente non modifichi il contenuto della lista.

Test Method Design: Il test crea una lista con due elementi, tenta di rimuovere un terzo elemento assente e poi controlla che dimensione e ordine degli elementi siano rimasti invariati.

Test Description: La lista contiene "A" e "B". Viene chiamato remove("C"), ma "C" non è presente. Il test assicura che la lista non venga alterata da una rimozione non riuscita.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene gli elementi "A" e "B".

Expected result: size() == 2, get(0) == "A", get(1) == "B".

- **public void testRemoveNullElement()**

Summary: Verifica che `remove(Object o)` gestisca correttamente la rimozione di un elemento null.

Test Method Design: Il test inserisce un elemento normale e un valore null, poi richiama `remove(null)` e controlla che null non sia più presente.

Test Description: La lista contiene "A" e null. Dopo la rimozione di null, la lista deve contenere solo "A", e il valore null non deve essere più rilevato da `contains(null)`.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: `remove(null) == true`, `contains(null) == false`, `size() == 1`.

- **public void testRemoveNullNotPresent()**

Summary: Verifica che `remove(null)` restituisca false se la lista non contiene valori null.

Test Method Design: Il test inserisce un elemento non nullo e tenta di rimuovere null.

Test Description: La lista contiene solo "A". Poiché null non è presente, la rimozione non deve avere effetto e deve restituire false.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" (invariata dopo `remove`).

Expected result: `remove(null) == false`, `size() == 1`.

- **ListAdapter.clear()**

- **public void testClearEmptyList()**

Summary: Verifica che il metodo `clear()` possa essere chiamato correttamente anche su una lista già vuota.

Test Method Design: Il test richiama `clear()` su una lista appena creata, senza inserire prima alcun elemento. Successivamente vengono controllati lo stato di vuotezza e la dimensione della lista.

Test Description: Il metodo `clear()` ha il compito di rimuovere tutti gli elementi presenti nella lista. In questo caso la lista è già vuota, quindi l'operazione non deve produrre modifiche effettive, ma deve comunque essere eseguibile senza generare errori. Il test verifica quindi che `clear()` gestisca correttamente anche il caso limite della lista priva di elementi.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: `size() == 0`, `isEmpty() == true`.

- **public void testClearRemovesAllElements()**

Summary: Verifica che `clear()` rimuova tutti gli elementi presenti nella lista.

Test Method Design: Il test inserisce più elementi nella lista tramite `add(Object o)`, richiama `clear()` e poi controlla che la lista risulti vuota e che gli elementi precedentemente inseriti non siano più contenuti.

Test Description: La lista viene inizialmente popolata con gli elementi "A", "B" e "C". Dopo la chiamata a `clear()`, la lista deve perdere ogni riferimento agli elementi inseriti e deve tornare nello stato di lista vuota. Il test controlla quindi sia la dimensione sia l'assenza dei singoli elementi.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista non contiene più alcun elemento.

Expected result: `size() == 0`, `isEmpty() == true`, `contains("A") == false`, `contains("B") == false`, `contains("C") == false`.

- **public void testClearNowEmptyList()**

Summary: Verifica che clear() funzioni correttamente anche su una lista che è diventata vuota tramite precedenti operazioni di rimozione.

Test Method Design: Il test inserisce alcuni elementi nella lista, li rimuove tutti tramite remove(Object o) e poi richiama clear().

Test Description: La lista non è vuota fin dall'inizio, ma viene portata allo stato vuoto attraverso una sequenza di rimozioni. Il test serve a controllare che clear() sia sicuro anche quando viene chiamato su una lista già svuotata da operazioni precedenti, senza provocare errori o modifiche anomale.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista è vuota (la chiamata di clear() lascia lo stato invariato).

Expected result: clear() termina senza lanciare eccezioni.

- **public void testClearAfterMixedOperations()**

Summary: Verifica che clear() svuoti correttamente la lista anche dopo una sequenza mista di inserimenti e rimozioni.

Test Method Design: Il test inserisce tre elementi, ne rimuove uno e successivamente richiama clear(). Alla fine vengono controllate la dimensione, la condizione di lista vuota e l'assenza degli elementi rimasti prima dello svuotamento.

Test Description: Dopo l'inserimento di "A", "B" e "C", viene rimosso "B". La lista contiene quindi ancora "A" e "C". Il test verifica che clear() rimuova anche gli elementi residui, riportando la lista allo stato vuoto indipendentemente dalle operazioni precedenti.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista è vuota.

Expected result: size() == 0, isEmpty() == true, contains("A") == contains("B") == contains("C") == false.

- **ListAdapter.contains(Object o)**

- **public void testContainsEmptyList()**

Summary: Verifica che contains(Object o) restituisca false quando viene chiamato su una lista vuota.

Test Method Design: Il test richiama contains("A") su una lista appena creata, senza effettuare inserimenti precedenti.

Test Description: Poiché la lista non contiene alcun elemento, nessun oggetto può risultare presente al suo interno. Il test controlla quindi il comportamento del metodo nel caso limite della lista vuota.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota (contains() non fa modifiche strutturali).

Expected result: contains("A") == false.

- **public void testContainsExistingElement()**

Summary: Verifica che contains(Object o) restituisca true quando l'elemento cercato è presente nella lista.

Test Method Design: Il test inserisce un elemento nella lista tramite add(Object o) e poi richiama contains() sullo stesso elemento.

Test Description: Dopo l'inserimento di "A", il metodo contains("A") deve riconoscere correttamente la presenza dell'elemento. Il test verifica quindi il caso standard di ricerca positiva.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene l'elemento "A".

Expected result: contains("A") == true.

- **public void testContainsNotExistingElement()**

Summary: Verifica che contains(Object o) restituisca false quando l'elemento cercato non è presente nella lista, che non è vuota.

Test Method Design: Il test inserisce "A" nella lista e poi cerca "B" tramite contains().

Test Description: La lista contiene un elemento diverso da quello cercato. Il metodo deve quindi distinguere correttamente tra elemento presente ed elemento assente, restituendo false per "B".

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene solo l'elemento "A".

Expected result: contains("B") == false.

- **public void testContainsMoreElements()**

Summary: Verifica che contains(Object o) funzioni correttamente anche quando la lista contiene più elementi.

Test Method Design: Il test inserisce due elementi distinti nella lista e controlla separatamente la presenza di entrambi.

Test Description: Gli elementi "A" e "B" vengono aggiunti alla lista. Il test verifica che contains() sia in grado di trovare ciascun elemento presente, indipendentemente dalla sua posizione nella lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene gli elementi "A" e "B".

Expected result: contains("A") == contains("B") == true.

- **public void testContainsNullElement()**

Summary: Verifica che contains(Object o) gestisca correttamente la ricerca di un elemento null presente nella lista.

Test Method Design: Il test inserisce null nella lista e poi richiama contains(null).

Test Description: Il valore null viene trattato come un elemento valido della lista. Il test controlla quindi che il metodo non generi errori nel confronto con null e che riesca a riconoscerne la presenza.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene un elemento null.

Expected result: contains(null) == true.

- **public void testContainsNullNotPresent()**

Summary: Verifica che contains(null) restituisca false se la lista non contiene valori null.

Test Method Design: Il test inserisce un elemento non nullo nella lista e poi richiama contains(null).

Test Description: La lista contiene "A" ma non contiene alcun elemento null. Il test controlla che il metodo non confonda l'assenza di null con la presenza di altri elementi e restituisca correttamente false.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene solo l'elemento "A".

Expected result: contains(null) == false.

- **ListAdapter.equals(Object o)**

- **public void testEqualsSameElements()**

Summary: Verifica che due liste contenenti gli stessi elementi nello stesso ordine siano considerate uguali.

Test Method Design: Il test crea due istanze distinte di ListAdapter, inserisce in entrambe gli stessi elementi nello stesso ordine e poi le confronta tramite equals(Object o).

Test Description: Le due liste contengono "A", "B" e "C". Anche se sono due oggetti diversi in memoria, devono essere considerate uguali perché il confronto tra liste si basa sugli elementi contenuti e sul loro ordine.

Pre-Condition: list è appena stata creata e non contiene elementi.

Post-Condition: Entrambe le liste contengono gli stessi elementi nello stesso ordine.

Expected result: list.equals(other) == true.

- **public void testEqualsReverse()**

Summary: Verifica che il metodo equals(Object o) rispetti la proprietà di simmetria (se list è uguale a other, anche other dovrà essere uguale a list).

Test Method Design: Il test crea due liste con lo stesso contenuto e verifica il confronto in entrambe le direzioni.

Test Description: Se list.equals(other) restituisce true, allora anche other.equals(list) deve restituire true. Il test controlla quindi che il risultato dell'uguaglianza non dipenda dall'ordine con cui vengono confrontati i due oggetti.

Pre-Condition: list è appena stata creata e non contiene elementi.

Post-Condition: Le due liste contengono gli stessi elementi nello stesso ordine e non vengono modificate dal confronto.

Expected result: list.equals(other) == other.equals(list) == true.

- **public void testEqualsEmptyLists()**

Summary: Verifica che due liste vuote siano considerate uguali.

Test Method Design: Il test crea una seconda istanza vuota di ListAdapter e la confronta con la lista inizializzata dal metodo setUp().

Test Description: Due liste vuote hanno la stessa dimensione e non contengono elementi differenti. Per questo motivo devono essere considerate uguali dal metodo equals(). La lista vuota viene considerato caso limite

Pre-Condition: list è appena stata creata e non contiene elementi.

Post-Condition: Entrambe le liste rimangono vuote.

Expected result: list.equals(other) == true.

- **public void testEqualsWithItself()**

Summary: Verifica che una lista sia uguale a se stessa.

Test Method Design: Il test inserisce alcuni elementi nella lista e poi richiama equals() passando come parametro la stessa istanza.

Test Description: Ogni oggetto deve essere uguale a se stesso. Il test controlla quindi il caso riflessivo del metodo equals(), confrontando la lista con il proprio riferimento.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene gli elementi inseriti e non viene modificata dal confronto.

Expected result: list.equals(list) == true.

- **public void testEqualsNull()**

Summary: Verifica che una lista non sia considerata uguale a null.

Test Method Design: Il test inserisce un elemento nella lista e richiama equals(null).

Test Description: Il confronto con null deve essere gestito correttamente e non deve generare eccezioni. Secondo il contratto generale di equals(), un oggetto non nullo deve risultare diverso da null.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene l'elemento "A" e rimane invariata dopo il confronto.

Expected result: equals(null) == false.

- **public void testEqualsDifferentType()**

Summary: Verifica che una lista non sia considerata uguale a un oggetto di tipo diverso.

Test Method Design: Il test inserisce "A" nella lista e poi confronta la lista con la stringa "A".

Test Description: Anche se il valore contenuto nella lista è uguale alla stringa usata nel confronto, una lista non deve essere considerata uguale a un singolo oggetto di tipo String. Il metodo deve quindi verificare anche la compatibilità del tipo dell'oggetto confrontato.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: equals("A") == false.

- **public void testEqualsDifferentSize()**

Summary: Verifica che due liste con dimensione diversa non siano considerate uguali.

Test Method Design: Il test crea due liste con alcuni elementi comuni, ma con numero totale di elementi diverso.

Test Description: La prima lista contiene "A" e "B", mentre la seconda contiene "A", "B" e "C". Anche se una parte iniziale del contenuto coincide, le liste non possono essere uguali perché hanno dimensione diversa.

Pre-Condition: list è appena stata creata e non contiene elementi.

Post-Condition: Le due liste contengono un numero diverso di elementi.

Expected result: list.equals(other) == false.

- **public void testEqualsDifferentElements()**

Summary: Verifica che due liste della stessa dimensione ma con almeno un elemento diverso non siano considerate uguali.

Test Method Design: Il test crea due liste di uguale dimensione e cambia un elemento nella seconda lista.

Test Description: La prima lista contiene "A", "B" e "C", mentre la seconda contiene "A", "X" e "C". Le due liste hanno la stessa dimensione, ma il contenuto non coincide in tutte le posizioni.

Pre-Condition: list è appena stata creata e non contiene elementi.

Post-Condition: Le due liste hanno stessa dimensione ma contenuto diverso.

Expected result: list.equals(other) == false.

- **public void testEqualsDifferentOrder()**

Summary: Verifica che l'ordine degli elementi sia rilevante nel confronto tra liste.

Test Method Design: Il test crea due liste contenenti gli stessi elementi, ma inseriti in ordine diverso.

Test Description: Una lista contiene "A" e "B", mentre l'altra contiene "B" e "A". Anche se gli elementi sono gli stessi, l'ordine è diverso; poiché una lista è una struttura ordinata, le due liste non devono essere considerate uguali.

Pre-Condition: list è appena stata creata e non contiene elementi.

Post-Condition: Le due liste contengono gli stessi elementi ma in ordine differente.

Expected result: list.equals(other) == false.

- **public void testEqualsNullElement()**

Summary: Verifica che equals(Object o) gestisca correttamente la presenza di elementi null.

Test Method Design: Il test crea due liste con gli stessi elementi e con un valore null nella stessa posizione.

Test Description: Entrambe le liste contengono "A", null e "C". Il valore null deve essere considerato correttamente nel confronto: se compare nella stessa posizione in entrambe le liste, le due liste devono risultare uguali.

Pre-Condition: list è appena stata creata e non contiene elementi.

Post-Condition: Le due liste contengono gli stessi elementi, compreso null, nello stesso ordine.

Expected result: list.equals(other) == true.

- **public void testEqualsNullElement1()**

Summary: Verifica che due liste non siano uguali se null è presente solo in una delle due.

Test Method Design: Il test crea due liste con la stessa dimensione, ma nella prima viene inserito null mentre nella seconda, nella stessa posizione, viene inserito un elemento non nullo.

Test Description: La prima lista contiene "A", null e "C", mentre la seconda contiene "A", "B" e "C". Anche se le liste hanno stessa dimensione e coincidono in alcune posizioni, differiscono nell'elemento centrale, quindi non devono essere considerate uguali.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: Le due liste hanno stessa dimensione ma differiscono in una posizione.

Expected result: list.equals(other) == false.

- **ListAdapter.hashCode()**

- **public void testHashCodeEqualLists()**

Summary: Verifica che due liste uguali abbiano lo stesso valore di hashCode() (in questo, come in quelli successivi, si usano esempi con due liste solo perché considerati caso emblematico del confronto di hash tra più list).

Test Method Design: Il test crea due liste uguali, controlla che equals() restituisca true e poi confronta gli hash code.

Test Description: Viene controllata la coerenza tra equals() e hashCode().

Pre-Condition: list è appena stata creata e non contiene elementi.

Post-Condition: Entrambe le liste contengono “A”, “B”, “C”.

Expected result: list.equals(other) == true, list.hashCode() == other.hashCode().

- **public void testHashCodeEmptyLists()**

Summary: Verifica che due liste vuote abbiano lo stesso hash code.

Test Method Design: Il test crea una seconda lista vuota, oltre a quella di setUp, e confronta il valore di hashCode() delle due liste.

Test Description: Viene controllato che lo stato vuoto produca un hash code coerente tra liste equivalenti. E’ un test specifico del caso “limite” delle due liste vuote.

Pre-Condition: list è appena stata creata e non contiene elementi.

Post-Condition: Entrambe le liste sono vuote.

Expected result: list.hashCode() == other.hashCode().

- **public void testHashCodeNullElement()**

Summary: Verifica che hashCode() gestisca correttamente liste contenenti elementi null.

Test Method Design: Il test crea due liste uguali, contenenti almeno un elemento null, e confronta i rispettivi hashCode.

Test Description: Viene controllato che la presenza di null, considerato caso potenzialmente critico, non impedisca il calcolo coerente dell’hash code.

Pre-Condition: list è appena stata creata e non contiene elementi.

Post-Condition: Entrambe le liste contengono “A”, null, “C”.

Expected result: list.hashCode() == other.hashCode().

- **public void testHashCodeAfterModify()**

Summary: Verifica che l'hash code cambi dopo una modifica strutturale della lista.

Test Method Design: Il test salva l'hash code iniziale della lista vuota, inserisce "A" e confronta il nuovo hash code con quello precedente.

Test Description: Il test confronta il valore di hashCode() prodotto da due stati diversi della stessa lista: prima quando la lista è vuota e poi dopo l'inserimento dell'elemento "A". In questo modo viene verificato che, per i dati utilizzati nel test, il calcolo dell'hash code tenga conto del contenuto effettivo della lista e produca un valore coerente con la modifica effettuata.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene l'elemento "A".

Expected result: dopo la modifica strutturale: `oldHashCode != list.hashCode(), list.hashCode() == 96`.

ListAdapterCollectionTest

Summary

Il test case ListAdapterCollectionTest verifica il corretto funzionamento dei metodi della classe ListAdapter che operano su un'intera collection.

L'obiettivo è controllare che la lista gestisca correttamente le operazioni di confronto, inserimento, rimozione e mantenimento degli elementi a partire da un'altra collection.

In particolare, vengono testati i metodi:

- containsAll(HCollection c);
- addAll(HCollection c);
- addAll(int index, HCollection c);
- removeAll(HCollection c);
- retainAll(HCollection c).

I test considerano sia casi ordinari, sia casi limite, come collection vuota, collection nulla, elementi null, elementi duplicati, indici non validi e operazioni eseguite passando la lista stessa come parametro.

Test Case Design

Ogni test crea due nuove istanze di ListAdapter, inizializzate prima dell'esecuzione tramite il metodo setUp().

La variabile list rappresenta la lista principale su cui viene eseguito il metodo da testare, mentre la variabile other rappresenta la collection passata come parametro (viene utilizzato un altro ListAdapter perché la classe implementa HCollection, rimanendo quindi coerente con il contesto del progetto).

I test sono organizzati per metodo testato. Per ogni metodo vengono verificati sia il valore restituito, quando previsto, sia gli effetti prodotti sul contenuto della lista.

Le verifiche controllano quindi la dimensione finale, l'ordine degli elementi, la presenza o assenza di determinati oggetti, la gestione di null e il corretto lancio delle eccezioni nei casi non validi.

Test

- **ListAdapter.containsAll(HCollection c)**

- **public void testContainsAllEmptyCollection()**

Summary: Verifica che containsAll(HCollection c) restituisca true quando la collection passata come parametro è vuota.

Test Method Design: Il test inserisce alcuni elementi nella lista principale e lascia vuota la collection other. Successivamente richiama containsAll(other).

Test Description: Una lista deve sempre contenere tutti gli elementi di una collection vuota, poiché non esiste alcun elemento mancante da verificare. Il test controlla quindi il comportamento del metodo nel caso limite in cui la collection passata non contenga elementi.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A" e "B", other è vuota.

Expected result: containsAll(other) == true.

- **public void testContainsAllEqualElements()**

Summary: Verifica che containsAll(HCollection c) restituisca true quando lista e collection contengono gli stessi elementi.

Test Method Design: Il test inserisce gli elementi "A", "B" e "C" sia nella lista principale sia nella collection other, poi richiama containsAll(other).

Test Description: Il metodo deve riconoscere che tutti gli elementi presenti nella collection passata come parametro sono contenuti anche nella lista principale. Il test rappresenta il caso standard in cui le due collection hanno lo stesso contenuto.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list e other sono uguali e contengono "A", "B", "C".

Expected result: containsAll(other) == true.

- **public void testContainsAllAllElementsPresent()**

Summary: Verifica che containsAll(HCollection c) restituisca true quando tutti gli elementi della collection sono presenti nella lista, anche se la lista contiene elementi aggiuntivi.

Test Method Design: Il test inserisce quattro elementi nella lista principale e solo tre nella collection other.

Test Description: La lista principale contiene tutti gli elementi richiesti da other, più un ulteriore elemento. Il metodo non deve richiedere che le due collection siano uguali, ma solo che ogni elemento di other sia presente nella lista.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A", "B", "C", "D". other contiene "A", "B", "C".

Expected result: containsAll(other) == true.

- **public void testContainsAllWithMissingElement()**

Summary: Verifica che containsAll(HCollection c) restituisca false se almeno un elemento della collection non è presente nella lista.

Test Method Design: Il test inserisce nella lista principale "A" e "B", mentre nella collection other inserisce "A" e "C".

Test Description: Anche se un elemento della collection è presente nella lista, il metodo deve restituire false perché "C" non è contenuto nella lista principale. Il test controlla quindi il caso di ricerca parzialmente fallita.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A" e "B". other contiene "A" e "C".

Expected result: containsAll(other) == false.

- **public void testContainsAllWithNullElement()**

Summary: Verifica che containsAll(HCollection c) gestisca correttamente un elemento null presente nella lista.

Test Method Design: Il test inserisce null nella lista principale e nella collection other, poi richiama containsAll(other).

Test Description: Il valore null deve essere trattato come un elemento valido della lista. Il test controlla quindi che il metodo sia in grado di riconoscere la presenza di null senza generare errori.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A", null, "C". other contiene null.

Expected result: containsAll(other) == true.

- **public void testContainsAllWithNullNotPresent()**

Summary: Verifica che containsAll(HCollection c) restituisca false quando la collection contiene null, ma la lista principale no.

Test Method Design: Il test inserisce solo elementi non nulli nella lista principale e inserisce null nella collection other.

Test Description: Il metodo deve distinguere correttamente tra presenza e assenza di null. Poiché la lista non contiene valori nulli, la ricerca deve fallire.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A" e "B". other contiene "null".

Expected result: containsAll(other) == false.

- **public void testContainsAllWithItself()**

Summary: Verifica che containsAll(HCollection c) restituisca true quando la lista viene confrontata con se stessa.

Test Method Design: Il test inserisce alcuni elementi nella lista e poi richiama list.containsAll(list).

Test Description: Una lista deve necessariamente contenere tutti i propri elementi. Il test controlla quindi il caso in cui il parametro passato sia lo stesso oggetto su cui viene invocato il metodo.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: La lista contiene "A" e "B".

Expected result: list.containsAll(list) == true.

- **public void testContainsAllWithNullCollection()**

Summary: Verifica che containsAll(HCollection c) lanci NullPointerException quando la collection passata è nulla.

Test Method Design: Il test richiama containsAll(null) sulla lista principale.

Test Description: Il metodo richiede una collection valida come parametro. Passare null rappresenta un uso non corretto del metodo e deve quindi produrre l'eccezione prevista.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: Entrambe le liste rimangono vuote.

Expected result: containsAll(null) lancia NullPointerException.

ListAdapter.addAll(HCollection c)

- public void testAddAllWithEmptyCollection()

Summary: Verifica che addAll(HCollection c) restituisca false quando la collection passata è vuota.

Test Method Design: Il test inserisce un elemento nella lista principale e lascia vuota la collection other. Poi richiama addAll(other).

Test Description: Poiché la collection da aggiungere non contiene elementi, la lista principale non deve subire modifiche. Il metodo deve quindi indicare che non è avvenuta alcuna modifica strutturale.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A". other rimane vuota.

Expected result: addAll(other) == false, size() == 1, get(0) == "A".

- public void testAddAllAddsElementsAtEnd()

Summary: Verifica che addAll(HCollection c) aggiunga tutti gli elementi della collection in coda alla lista.

Test Method Design: Il test inserisce "A" nella lista principale e "B", "C" nella collection other. Successivamente richiama addAll(other).

Test Description: Gli elementi della collection devono essere aggiunti dopo quelli già presenti nella lista, mantenendo l'ordine originario della collection aggiunta.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A", "B" e "C". other contiene "B", "C".

Expected result: addAll(other) == true, size() == 3, get(0) == "A", get(1) == "B", get(2) == "C".

- public void testAddAllKeepsOrder()

Summary: Verifica che addAll(HCollection c) mantenga l'ordine degli elementi aggiunti.

Test Method Design: Il test inserisce "A", "B" e "C" nella collection other, poi richiama list.addAll(other) su una lista inizialmente vuota.

Test Description: Il metodo deve copiare gli elementi rispettando l'ordine di iterazione della collection passata. Il test controlla quindi che gli elementi risultino nella lista nello stesso ordine con cui erano presenti in other.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: Le due liste contengono gli stessi elementi e nello stesso ordine ("A", "B", "C").

Expected result: get(0) == "A", get(1) == "B", get(2) == "C".

- **public void testAddAllWithNullElement()**

Summary: Verifica che `addAll(HCollection c)` gestisca correttamente l'inserimento di elementi null.

Test Method Design: Il test inserisce nella collection `other` gli elementi "A", null e "C", poi li aggiunge alla lista principale.

Test Description: Il valore null deve essere aggiunto come un normale elemento della lista. Il test controlla quindi che venga mantenuta anche la posizione del valore nullo all'interno della sequenza inserita.

Pre-Condition: Le liste `list` e `other` sono vuote.

Post-Condition: Entrambe le liste contengono "A", "null", "C".

Expected result: `size() == 3`, `get(0) == "A"`, `get(1) == null`, `get(2) == "C"`.

- **public void testAddAllWithItself()**

Summary: Verifica che `addAll(HCollection c)` funzioni correttamente quando la collection passata è la lista stessa.

Test Method Design: Il test inserisce "A" e "B" nella lista e poi richiama `list.addAll(list)`.

Test Description: Il metodo deve aggiungere alla fine della lista una copia degli elementi già presenti, senza entrare in comportamenti anomali dovuti al fatto che sorgente e destinazione coincidono.

Pre-Condition: Le liste `list` e `other` sono vuote.

Post-Condition: La lista contiene "A", "B", "A", "B". `other` rimane vuota.

Expected result: `addAll(list) == true`. La lista finale deve essere: "A", "B", "A", "B" (verificato con il metodo `get(index)`).

- **public void testAddAllWithNullCollection()**

Summary: Verifica che `addAll(HCollection c)` lanci `NullPointerException` quando la collection passata è nulla.

Test Method Design: Il test richiama `list.addAll(null)`.

Test Description: Il metodo necessita di una collection valida da cui leggere gli elementi da aggiungere. Il passaggio di un parametro nullo rappresenta un caso non valido.

Pre-Condition: Le liste `list` e `other` sono vuote.

Post-Condition: Le liste rimangono vuote.

Expected result: `addAll(null)` deve lanciare `NullPointerException`.

- **ListAdapter.addAll(int index, HCollection c)**
 - **public void testAddAllAtBeginning()**

Summary: Verifica che addAll(int index, HCollection c) inserisca correttamente gli elementi all'inizio della lista.

Test Method Design: Il test inserisce "C" nella lista principale e "A", "B" nella collection other. Poi richiama addAll(0, other).

Test Description: Gli elementi della collection devono essere inseriti a partire dall'indice specificato. In questo caso l'indice 0 corrisponde all'inizio della lista, quindi gli elementi preesistenti devono essere spostati in avanti.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A", "B", "C". other contiene: "A" e "B".

Expected result: addAll(0, other) == true. list contiene, nell'ordine: "A", "B", "C" (verificato con il metodo get(index)).
 - **public void testAddAllAtMiddle()**

Summary: Verifica che addAll(int index, HCollection c) inserisca correttamente gli elementi in una posizione intermedia.

Test Method Design: Il test crea una lista con "A" e "D", poi inserisce "B" e "C" dalla collection other all'indice 1.

Test Description: Gli elementi di other devono essere inseriti tra gli elementi già presenti nella lista, senza perdere né alterare quelli originari. Il test controlla quindi lo spostamento corretto degli elementi successivi all'indice di inserimento.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A", "B", "C", "D": other contiene "B" e "C"

Expected result: addAll(1, other) == true. list contiene nell'ordine: "A", "B", "C", "D" (verificato con il metodo get(index)).
 - **public void testAddAllAtEnd()**

Summary: Verifica che addAll(int index, HCollection c) funzioni correttamente quando l'indice è uguale alla dimensione della lista.

Test Method Design: Il test inserisce "A" nella lista principale, "B" e "C" in other, poi richiama addAll(1, other).

Test Description: Quando l'indice è uguale a size(), l'inserimento deve avvenire in coda. Il test controlla quindi che questo caso venga gestito come un normale inserimento finale.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A", "B", "C". other contiene "B", "C".

Expected result: addAll(1, other) == true. list contiene nell'ordine: "A", "B", "C" (verificato con il metodo get(index)).

- **public void testAddAllAtIndexWithEmptyCollection()**

Summary: Verifica che `addAll(int index, HCollection c)` restituisca false quando la collection da inserire è vuota.

Test Method Design: Il test crea una lista con "A" e "B" e richiama `addAll(1, other)` con other vuota.

Test Description: Anche se l'indice è valido, non essendoci elementi da aggiungere la lista non deve essere modificata. Il metodo deve quindi restituire false.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A" e "B". other rimane vuota.

Expected result: `list.addAll(1, other) == false`, `list.size() == 2`, `list.get(0) == "A"`, `list.get(1) == "B"`.

- **public void testAddAllAtIndexWithNullElement()**

Summary: Verifica che `addAll(int index, HCollection c)` gestisca correttamente l'inserimento di un elemento null.

Test Method Design: Il test crea una lista con "A" e "D" e inserisce all'indice 1 una collection contenente "B", null e "C".

Test Description: Il valore null deve essere inserito nella posizione corretta insieme agli altri elementi della collection. Il test controlla che l'ordine complessivo venga mantenuto.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A", "B", null, "C", "D". other contiene "B", null, "C".

Expected result: list contiene, nell'ordine: "A", "B", null, "C", "D" (verificato con il metodo `get(index)`).

- **public void testAddAllAtIndexWithItself()**

Summary: Verifica che `addAll(int index, HCollection c)` funzioni correttamente quando la collection passata è la lista stessa.

Test Method Design: Il test inserisce "A", "B" e "C" nella lista e poi richiama `list.addAll(1, list)`.

Test Description: Il metodo deve inserire una copia degli elementi della lista a partire dall'indice indicato, senza alterare in modo scorretto la sequenza originaria durante l'operazione.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A", "A", "B", "C", "B", "C".

Expected result: list contiene, nell'ordine: "A", "A", "B", "C", "B", "C".

- **public void testAddAllAtNegativeIndex()**

Summary: Verifica che `addAll(int index, HCollection c)` lanci `IndexOutOfBoundsException` con indice negativo.

Test Method Design: Il test inserisce un elemento in `other` e richiama `list.addAll(-1, other)`.

Test Description: Un indice negativo non rappresenta una posizione valida della lista. Il metodo deve quindi impedire l'operazione e lanciare l'eccezione prevista.

Pre-Condition: Le liste `list` e `other` sono vuote.

Post-Condition: `list` rimane vuota, `other` contiene "A".

Expected result: `addAll(-1, other)` deve lanciare `IndexOutOfBoundsException`.

- **public void testAddAllAtIndexGreaterThanSize()**

Summary: Verifica che `addAll(int index, HCollection c)` lanci `IndexOutOfBoundsException` quando l'indice è maggiore della dimensione della lista.

Test Method Design: Il test crea una lista di dimensione 1 e prova a inserire una collection all'indice 2.

Test Description: Per l'inserimento è ammesso un indice compreso tra 0 e `size()` inclusi. Un indice maggiore di `size()` non corrisponde ad alcuna posizione valida.

Pre-Condition: Le liste `list` e `other` sono vuote.

Post-Condition: `list` contiene "A", `other` contiene "B"

Expected result: `addAll(2, other)` deve lanciare `IndexOutOfBoundsException`

- **public void testAddAllAtIndexWithNullCollection()**

Summary: Verifica che `addAll(int index, HCollection c)` lanci `NullPointerException` quando la collection passata è nulla.

Test Method Design: Il test richiama `list.addAll(0, null)`.

Test Description: Anche se l'indice è valido, il metodo non può leggere elementi da una collection nulla. Il parametro non valido deve quindi generare l'eccezione prevista.

Pre-Condition: Le liste `list` e `other` sono vuote.

Post-Condition: Le liste rimangono vuote.

Expected result: `addAll(0, null)` deve lanciare `NullPointerException`.

- **ListAdapter.removeAll(HCollection c)**

- **public void testRemoveAllRemovesElements()**

Summary: Verifica che removeAll(HCollection c) rimuova dalla lista gli elementi presenti nella collection.

Test Method Design: Il test inserisce "A", "B" e "C" nella lista principale e "A", "C" nella collection other.

Test Description: Il metodo deve eliminare dalla lista tutti gli elementi che sono contenuti nella collection passata. In questo caso devono essere rimossi "A" e "C", lasciando solo "B".

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "B", other contiene "A", "C".

Expected result: removeAll(other) == true. size() == 1, get(0) == "B".

- **public void testRemoveAllWithNoCommonElements()**

Summary: Verifica che removeAll(HCollection c) restituisca false quando non ci sono elementi comuni da rimuovere.

Test Method Design: Il test crea una lista con "A" e "B" e una collection other con "C" e "D".

Test Description: Poiché nessun elemento della lista è contenuto nella collection passata, la lista non deve essere modificata. Il metodo deve quindi restituire false.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A" e "B". other contiene "C" e "D".

Expected result: removeAll(other) == false. size() == 2, get(0) == "A", get(1) == "B".

- **public void testRemoveAllRemovesAllOccurrences()**

Summary: Verifica che removeAll(HCollection c) rimuova tutte le occorrenze degli elementi presenti nella collection.

Test Method Design: Il test inserisce nella lista due occorrenze di "A" e inserisce "A" nella collection other.

Test Description: Il metodo non deve limitarsi a rimuovere la prima occorrenza dell'elemento, ma deve eliminare tutte le occorrenze presenti nella lista. Il test controlla quindi la gestione dei duplicati.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "B" e "C". other contiene "A".

Expected result: removeAll(other) == true, size() == 2, get(0) == "B", get(1) == "C".

- **public void testRemoveAllWithNullElement()**

Summary: Verifica che removeAll(HCollection c) rimuova correttamente un elemento null.

Test Method Design: Il test inserisce null nella lista principale e nella collection other, poi richiama removeAll(other).

Test Description: Il valore null deve essere considerato un elemento valido anche nelle operazioni di rimozione collettiva. Il test controlla che venga eliminato solo il valore nullo, lasciando invariati gli altri elementi.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A" e "B". other contiene null.

Expected result: removeAll(other) == true, size() == 2, get(0) == "A", get(1) == "B".

- **public void testRemoveAllWithEmptyCollection()**

Summary: Verifica che removeAll(HCollection c) restituisca false quando la collection passata è vuota.

Test Method Design: Il test crea una lista con "A" e "B" e lascia vuota la collection other.

Test Description: Una collection vuota non contiene elementi da rimuovere dalla lista principale. Il metodo non deve quindi modificare la lista.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A", "B". other rimane vuota.

Expected result: removeAll(other) == false. size() == 2.

- **public void testRemoveAllWithItself()**

Summary: Verifica che removeAll(HCollection c) svuoti la lista quando viene passato come parametro la lista stessa.

Test Method Design: Il test inserisce "A", "B" e "C" nella lista, poi richiama list.removeAll(list).

Test Description: Poiché ogni elemento della lista è contenuto nella lista stessa, il metodo deve rimuovere tutti gli elementi. Il test controlla quindi il comportamento nel caso in cui sorgente di confronto e lista da modificare coincidano.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: Entrambe le liste rimangono vuote.

Expected result: removeAll(list) == true. size() == 0. isEmpty() == true.

- **public void testRemoveAllWithNullCollection()**

Summary: Verifica che `removeAll(HCollection c)` lanci `NullPointerException` quando la collection passata è nulla.

Test Method Design: Il test richiama `list.removeAll(null)`.

Test Description: Il metodo necessita di una collection valida per stabilire quali elementi rimuovere. Il passaggio di null deve quindi essere considerato non valido.

Pre-Condition: Le liste `list` e `other` sono vuote.

Post-Condition: Entrambe le liste rimangono vuote.

Expected result: `removeAll(null)` deve lanciare `NullPointerException`

- **ListAdapter.retainAll(HCollection c)**

- **public void testRetainAllKeepsOnlyCommonElements()**

Summary: Verifica che retainAll(HCollection c) mantenga nella lista solo gli elementi presenti nella collection passata.

Test Method Design: Il test inserisce "A", "B" e "C" nella lista principale e "A", "C" nella collection other.

Test Description: Il metodo deve rimuovere dalla lista tutti gli elementi che non sono contenuti nella collection passata. In questo caso "B" deve essere eliminato, mentre "A" e "C" devono rimanere.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: Entrambe le liste contengono "A" e "C".

Expected result: retainAll(other) == true; size() == 2; get(0) == "A"; get(1) == "C".

- **public void testRetainAllReturnsFalseWhenListDoesNotChange()**

Summary: Verifica che retainAll(HCollection c) restituisca false quando la lista non viene modificata.

Test Method Design: Il test crea una lista con "A" e "B" e una collection other che contiene "A", "B" e "C".

Test Description: Tutti gli elementi della lista sono già contenuti nella collection passata, quindi nessun elemento deve essere rimosso. Il metodo deve indicare che non è avvenuta alcuna modifica.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A", "B". other contiene "A", "B", "C".

Expected result: retainAll(other) == false. size() == 2; get(0) == "A"; get(1) == "B".

- **public void testRetainAllWithNoCommonElements()**

Summary: Verifica che retainAll(HCollection c) svuoti la lista se non esistono elementi comuni con la collection passata.

Test Method Design: Il test crea una lista con "A" e "B" e una collection other con "C" e "D".

Test Description: Nessun elemento della lista è contenuto nella collection passata. Di conseguenza, il metodo deve rimuovere tutti gli elementi della lista principale.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list rimane vuota. other contiene "C" e "D".

Expected result: retainAll(other) == true; size() == 0; isEmpty() == true.

- **public void testRetainAllWithNullElement()**

Summary: Verifica che retainAll(HCollection c) mantenga correttamente un elemento null quando è presente nella collection passata.

Test Method Design: Il test inserisce "A", null e "B" nella lista, mentre in other inserisce solo null.

Test Description: Il metodo deve conservare solo gli elementi contenuti nella collection passata. In questo caso l'unico elemento da mantenere è null, mentre gli altri devono essere rimossi.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: Entrambe le liste contengono solo null.

Expected result: retainAll(other) == true; size() == 1; get(0) == null.

- **public void testRetainAllWithEmptyCollection()**

Summary: Verifica che retainAll(HCollection c) svuoti la lista quando la collection passata è vuota.

Test Method Design: Il test crea una lista con "A" e "B" e richiama retainAll(other) con other vuota.

Test Description: Poiché la collection passata non contiene alcun elemento, nessun elemento della lista può essere mantenuto. Il metodo deve quindi svuotare completamente la lista.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: Entrambe le liste rimangono vuote.

Expected result: retainAll(other) == true; size() == 0; isEmpty() == true.

- **public void testRetainAllOnEmptyList()**

Summary: Verifica che retainAll(HCollection c) restituisca false quando viene chiamato su una lista vuota

Test Method Design: Il test lascia vuota la lista principale, inserisce "A" in other e richiama retainAll(other).

Test Description: Se la lista principale è già vuota, non ci sono elementi da rimuovere. Il metodo non deve modificare la lista e deve quindi restituire false.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list rimane vuota. other contiene "A".

Expected result: retainAll(other) == false; size() == 0; isEmpty() == true.

- **public void testRetainAllWithItself()**

Summary: Verifica che retainAll(HCollection c) non modifichi la lista quando viene passato come parametro la lista stessa.

Test Method Design: Il test inserisce "A", "B" e "C" nella lista e poi richiama list.retainAll(list).

Test Description: Ogni elemento della lista è contenuto nella lista stessa, quindi nessun elemento deve essere rimosso. Il test controlla che il metodo riconosca correttamente questo caso e restituisca false.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: list contiene "A", "B", "C". other rimane vuota.

Expected result: retainAll(list) deve restituire false; list deve contenere ancora "A", "B" e "C".

- **public void testRetainAllWithNullCollection()**

Summary: Verifica che retainAll(HCollection c) lanci NullPointerException quando la collection passata come parametro è null.

Test Method Design: Il test richiama retainAll(null) sulla lista principale.

Test Description: Il metodo retainAll(HCollection c) richiede una collection valida per stabilire quali elementi mantenere nella lista. Il passaggio di un parametro null rappresenta quindi un utilizzo non valido del metodo. Il test controlla che, in questa situazione, venga lanciata l'eccezione prevista dalla documentazione.

Pre-Condition: Le liste list e other sono vuote.

Post-Condition: Le liste rimangono vuote.

Expected result: retainAll(null) deve lanciare NullPointerException.

ListAdapterToArrayTest

Summary

Il test case ListAdapterToArrayTest verifica il corretto funzionamento dei metodi toArray() e toArray(Object[] a) della classe ListAdapter.

L'obiettivo è controllare che la lista venga convertita correttamente in array, mantenendo ordine, dimensione, elementi null e duplicati. Viene inoltre verificato che l'array restituito sia indipendente dalla lista, cioè che modificare l'array ottenuto non modifichi il contenuto della lista originale.

Per il metodo toArray(Object[] a), i test verificano il comportamento con array di dimensione esatta, maggiore o minore rispetto alla dimensione della lista, oltre al caso di array nullo. Si considera che, in questa implementazione, il metodo con Object[] non effettua controlli di runtime sul tipo specifico dell'array (come specificato nelle note informative), ma lavora direttamente con array di Object.

Test Case Design

Ogni test utilizza una nuova istanza di ListAdapter, inizializzata prima dell'esecuzione tramite il metodo setUp().

I test sono organizzati in due gruppi principali: il primo riguarda toArray(), mentre il secondo riguarda toArray(Object[] a).

Per ogni metodo vengono verificati il contenuto dell'array restituito, la sua dimensione, il mantenimento dell'ordine degli elementi e il comportamento nei casi limite. Nel caso di toArray(Object[] a), viene controllato anche se l'array passato come parametro viene riutilizzato oppure se viene creato un nuovo array, in base alla sua dimensione rispetto alla lista.

Test

- **ListAdapter.toArray()**

- **public void testToArrayOnEmptyList()**

Summary: Verifica che toArray() chiamato su una lista vuota restituisca un array vuoto.

Test Method Design: Il test richiama toArray() su una lista appena creata e controlla la lunghezza dell'array restituito.

Test Description: Poiché la lista non contiene elementi, l'array prodotto dal metodo deve avere lunghezza pari a zero. Il test controlla quindi il caso limite della conversione di una lista vuota.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: L'array restituito deve avere length == 0.

- **public void testToArrayReturnsAllElementsInOrder()**

Summary: Verifica che toArray() restituisca tutti gli elementi della lista nello stesso ordine.

Test Method Design: Il test inserisce nella lista gli elementi "A", "B" e "C", poi richiama toArray() e controlla il contenuto dell'array.

Test Description: Il metodo toArray() deve produrre un array che rappresenti correttamente il contenuto della lista. Poiché ListAdapter mantiene un ordine tra gli elementi, anche l'array risultante deve rispettare lo stesso ordine di inserimento.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "C".

Expected result: L'array deve avere lunghezza 3 e contenere "A", "B" e "C", in questo ordine (ordine verificato con [index]).

- **public void testToArrayWithNullElement()**

Summary: Verifica che toArray() gestisca correttamente la presenza di un elemento null.

Test Method Design: Il test inserisce nella lista "A", null e "C", poi richiama toArray().

Test Description: Il valore null deve essere trattato come un normale elemento della lista. Il metodo deve quindi copiarlo nell'array nella stessa posizione in cui si trova nella lista, senza generare errori.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", null e "C".

Expected result: L'array deve contenere "A", null e "C" in questo ordine (ordine verificato con [index]).

- **public void testToArrayIsIndependent()**

Summary: Verifica che l'array restituito da toArray() sia indipendente dalla lista originale.

Test Method Design: Il test inserisce "A" e "B" nella lista, richiama toArray(), modifica il primo elemento dell'array e poi controlla il contenuto della lista.

Test Description: Il metodo toArray() deve restituire un array separato dalla struttura interna della lista. Di conseguenza, modificare l'array restituito non deve modificare gli elementi presenti nella lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: l'array contiene "X" e "B" mentre la lista "A" e "B".

Expected result: Dopo aver modificato array[0], list.get(0) == "A".

- **public void testToArrayWithDuplicateElements()**

Summary: Verifica che toArray() copi correttamente anche elementi duplicati.

Test Method Design: Il test inserisce nella lista "A", "B" e di nuovo "A", poi richiama toArray().

Test Description: La lista ammette elementi duplicati, quindi anche l'array restituito deve contenere tutte le occorrenze presenti nella lista. Il metodo non deve eliminare né accorpare duplicati.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "A".

Expected result: L'array deve avere lunghezza 3 e contenere "A", "B", "A" nello stesso ordine.

- **ListAdapter.toArray(Object[] a)**

- **public void testToArrayObjSizeArray()**

Summary: Verifica che toArray(Object[] a) riutilizzi l'array passato quando questo ha dimensione esatta.

Test Method Design: Il test inserisce tre elementi nella lista e passa al metodo un array di Object di lunghezza 3.

Test Description: Quando l'array passato ha dimensione uguale alla dimensione della lista, il metodo deve riutilizzare lo stesso array e copiarvi dentro gli elementi della lista. Il test controlla sia che l'array restituito sia lo stesso oggetto passato, sia che il contenuto venga copiato correttamente.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "C".

Expected result: result deve essere lo stesso oggetto di input (di conseguenza deve contenere "A", "B", "C").

- **public void testToArrayObjLargerArray()**

Summary: Verifica che toArray(Object[] a) riutilizzi l'array passato quando questo è più grande della lista.

Test Method Design: Il test inserisce due elementi nella lista e passa un array di lunghezza 4 già contenente alcuni valori.

Test Description: Quando l'array passato è più grande della lista, il metodo deve copiare gli elementi della lista nelle prime posizioni dell'array e inserire null subito dopo l'ultimo elemento copiato. Gli elementi successivi dell'array, se presenti, non devono essere necessariamente modificati.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B". input contiene "A", "B", null, "W".

Expected result: result deve essere lo stesso oggetto di input; result[0] == "A"; result[1] == "B"; result[2] == null; result[3] == "W".

- **public void testToArrayObjSmallerArray()**

Summary: Verifica che venga creato un nuovo array quando quello passato è troppo piccolo.

Test Method Design: Il test inserisce tre elementi nella lista e passa al metodo un array di lunghezza 1.

Test Description: Se l'array passato non ha spazio sufficiente per contenere tutti gli elementi della lista, il metodo deve creare un nuovo array di dimensione adeguata. Il test controlla quindi che l'array restituito non sia lo stesso passato come parametro e che contenga tutti gli elementi della lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "C".

Expected result: result non deve essere lo stesso oggetto di input; result.length == 3; result contiene "A", "B" e "C" (ordine verificato con [index]).

- **public void testToArrayObjEmptyListEmptyArray()**

Summary: Verifica il comportamento di toArray(Object[] a) con lista vuota e array vuoto.

Test Method Design: Il test richiama toArray(Object[] a) su una lista vuota passando un array di lunghezza 0.

Test Description: Poiché la lista è vuota e l'array passato ha già dimensione adeguata, il metodo deve semplicemente restituire lo stesso array senza modificarne il contenuto.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: result deve essere lo stesso oggetto di input; result.length == 0.

- **public void testToArrayObjEmptyListLargerArray()**

Summary: Verifica il comportamento di toArray(Object[] a) con lista vuota e array più grande.

Test Method Design: Il test crea un array di lunghezza 3 già contenente valori e richiama toArray(Object[] a) su una lista vuota.

Test Description: Quando la lista è vuota ma l'array passato ha lunghezza maggiore di zero, il metodo deve riutilizzare lo stesso array e inserire null nella prima posizione, indicando che non ci sono elementi della lista copiati. Gli elementi successivi dell'array restano invariati.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: result deve essere lo stesso oggetto di input; result[0] == null; result[1] == "B"; result[2] == "C".

- **public void testToArrayObjNullElement()**

Summary: Verifica che `toArray(Object[] a)` copi correttamente anche elementi null.

Test Method Design: Il test inserisce nella lista "A", null e "C", poi passa al metodo un array di dimensione esatta.

Test Description: Il valore null deve essere copiato nell'array come un normale elemento della lista. Il test controlla che venga mantenuta la stessa posizione occupata nella lista originale.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", null e "C".

Expected result: result deve essere lo stesso oggetto di input; result contiene "A", null, "C" (ordine verificato con `[index]`).

- **public void testToArrayObjDuplicateElements()**

Summary: Verifica che `toArray(Object[] a)` copi correttamente anche elementi duplicati.

Test Method Design: Il test inserisce nella lista "A", "B" e "A", poi passa al metodo un array di dimensione esatta.

Test Description: Il metodo deve copiare nell'array tutte le occorrenze presenti nella lista. La presenza di elementi duplicati non deve alterare la dimensione dell'array né l'ordine degli elementi copiati.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "A".

Expected result: result deve essere lo stesso oggetto di input; result contiene "A", "B", "A" (ordine verificato con `[index]`).

- **public void testToArrayObjectIsIndependent()**

Summary: Verifica che l'array ottenuto da `toArray(Object[] a)` sia indipendente dalla lista.

Test Method Design: Il test inserisce "A" e "B" nella lista, richiama `toArray(Object[] a)`, modifica il primo elemento dell'array restituito e poi controlla il contenuto della lista.

Test Description: Anche quando il metodo utilizza un array passato come parametro, la modifica dell'array restituito non deve modificare la lista originale. Il test controlla quindi che la lista non condivida i propri elementi in modo strutturale con l'array.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B".

Expected result: Dopo `result[0] = "X"`, `list.get(0) == "A"`.

- **public void testToArrayObjNullObj()**

Summary: Verifica che `toArray(Object[] a)` lanci `NullPointerException` quando l'array passato è null.

Test Method Design: Il test inserisce un elemento nella lista e richiama `toArray(null)`.

Test Description: Il metodo `toArray(Object[] a)` richiede un array valido come parametro. Passare null rappresenta quindi un caso non valido e deve generare l'eccezione prevista. In questa implementazione non viene considerato il controllo di runtime sul tipo specifico dell'array, ma viene comunque verificato il caso di parametro nullo.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: `toArray(null)` deve lanciare `NullPointerException`.

ListAdapterIndexedOperationsTest

Summary

Il test case ListAdapterIndexedOperationsTest verifica il corretto funzionamento delle operazioni indicizzate della classe ListAdapter.

L'obiettivo è controllare che i metodi:

- get(int index);
- set(int index, Object element);
- add(int index, Object element);
- remove(int index);

gestiscano correttamente l'accesso, la sostituzione, l'inserimento e la rimozione degli elementi in posizioni specifiche della lista.

I test considerano sia casi ordinari, sia casi limite, come primo elemento, ultimo elemento, lista vuota, indici negativi, indici uguali alla dimensione della lista, indici maggiori della dimensione ed elementi null.

Test Case Design

Ogni test utilizza una nuova istanza di ListAdapter, inizializzata prima dell'esecuzione tramite il metodo setUp().

I test sono organizzati per metodo testato. Per ogni metodo vengono verificati sia il valore restituito, quando previsto, sia gli effetti prodotti sulla struttura della lista.

Le verifiche controllano quindi il contenuto degli elementi, la dimensione della lista, lo spostamento degli elementi successivi in caso di inserimento o rimozione e il corretto lancio di IndexOutOfBoundsException nei casi in cui l'indice non sia valido.

Test

- **ListAdapter.get(int index)**

- **public void testGetValidIndex()**

Summary: Verifica che `get(int index)` restituisca l'elemento presente nella posizione indicata.

Test Method Design: Il test inserisce nella lista gli elementi "A", "B" e "C", poi richiama `get(1)`.

Test Description: Il metodo `get(int index)` deve permettere l'accesso diretto all'elemento che si trova nell'indice specificato. In questo caso viene scelto un indice centrale per verificare che il metodo non restituisca semplicemente il primo o l'ultimo elemento della lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene gli elementi "A", "B" e "C".

Expected result: `get(1) == "B"`.

- **public void testGetFirstElement()**

Summary: Verifica che `get(int index)` restituisca correttamente il primo elemento della lista.

Test Method Design: Il test inserisce nella lista gli elementi "A" e "B", poi richiama `get(0)`.

Test Description: L'indice 0 rappresenta la prima posizione valida della lista. Il test controlla quindi che il metodo riesca ad accedere correttamente al primo elemento senza errori di gestione degli indici.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B".

Expected result: `get(0) == "A"`.

- **public void testGetLastElement()**

Summary: Verifica che `get(int index)` restituisca correttamente l'ultimo elemento della lista.

Test Method Design: Il test inserisce nella lista gli elementi "A", "B" e "C", poi richiama `get(2)`.

Test Description: L'ultimo elemento valido si trova in posizione `size() - 1`. Il test controlla che il metodo `get()` gestisca correttamente anche l'ultimo indice valido della lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "C".

Expected result: `get(2) == "C"`.

- **public void testGetNullElement()**

Summary: Verifica che `get(int index)` funzioni correttamente anche quando l'elemento nella posizione richiesta è null.

Test Method Design: Il test inserisce nella lista "A" e null, poi richiama `get(1)`.

Test Description: Il valore null deve essere gestito come un normale elemento della lista. Il test controlla quindi che il metodo `get()` restituisca correttamente null quando questo è effettivamente presente nella posizione indicata.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e null.

Expected result: `get(1) == null`.

- **public void testGetNegativeIndex()**

Summary: Verifica che `get(int index)` lanci `IndexOutOfBoundsException` quando l'indice è negativo.

Test Method Design: Il test inserisce un elemento nella lista e poi richiama `get(-1)`.

Test Description: Gli indici validi di una lista partono da 0. Un indice negativo non rappresenta quindi una posizione valida e deve generare l'eccezione prevista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: `get(-1)` deve lanciare `IndexOutOfBoundsException`.

- **public void testGetIndexEqualToSize()**

Summary: Verifica che `get(int index)` lanci `IndexOutOfBoundsException` quando l'indice è uguale alla dimensione della lista.

Test Method Design: Il test inserisce un elemento nella lista e richiama `get(1)`.

Test Description: In una lista di dimensione 1, l'unico indice valido è 0. L'indice uguale a `size()` è fuori dai limiti della lista, quindi il metodo deve lanciare l'eccezione prevista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: `get(1)` deve lanciare `IndexOutOfBoundsException`.

- **public void testGetIndexGreaterThenSize()**

Summary: Verifica che `get(int index)` lanci `IndexOutOfBoundsException` quando l'indice è maggiore della dimensione della lista.

Test Method Design: Il test inserisce un elemento nella lista e richiama `get(2)`.

Test Description: Un indice maggiore di `size()` non può corrispondere ad alcun elemento della lista. Il test controlla quindi che il metodo impedisca l'accesso a posizioni non valide.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: `get(2)` deve lanciare `IndexOutOfBoundsException`.

- **public void testGetOnEmptyList()**

Summary: Verifica che `get(int index)` lanci `IndexOutOfBoundsException` quando viene chiamato su una lista vuota.

Test Method Design: Il test richiama `get(0)` su una lista appena creata.

Test Description: In una lista vuota non esiste alcun indice valido. Anche l'indice 0, che normalmente rappresenta la prima posizione, deve quindi essere considerato fuori dai limiti.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: `get(0)` deve lanciare `IndexOutOfBoundsException`.

- **ListAdapter.set(int index, Object element)**

- **public void testSetValidIndex()**

Summary: Verifica che set(int index, Object element) sostituisca l'elemento nella posizione indicata.

Test Method Design: Il test inserisce "A", "B" e "C" nella lista, poi richiama set(1, "X").

Test Description: Il metodo set() deve sostituire l'elemento presente nell'indice specificato con il nuovo elemento passato come parametro. In questo caso viene modificata una posizione centrale della lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "X", "C".

Expected result: get(1) == "X".

- **public void testSetReturnsOldElement()**

Summary: Verifica che set(int index, Object element) restituisca l'elemento precedentemente presente nella posizione modificata.

Test Method Design: Il test inserisce "A" e "B", poi richiama set(1, "X") e salva il valore restituito.

Test Description: Oltre a modificare la lista, il metodo set() deve restituire il vecchio elemento sostituito. Il test controlla quindi sia il comportamento di modifica sia il valore di ritorno del metodo.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "X".

Expected result: Il valore restituito da set(1, "X") deve essere "B".

- **public void testSetDoesNotChangeSize()**

Summary: Verifica che set(int index, Object element) non modifichi la dimensione della lista.

Test Method Design: Il test inserisce due elementi nella lista, richiama set(0, "X") e poi controlla size().

Test Description: Il metodo set() sostituisce un elemento già esistente, ma non aggiunge né rimuove elementi. Per questo motivo la dimensione della lista deve rimanere invariata dopo l'operazione.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "X", "B"

Expected result: size() == 2.

- **public void testSetFirstElement()**

Summary: Verifica che set(int index, Object element) funzioni correttamente sul primo elemento della lista.

Test Method Design: Il test inserisce “A” e “B”, poi richiama set(0, “X”).

Test Description: Il test controlla che il metodo riesca a modificare correttamente l’elemento in posizione iniziale, senza alterare gli elementi successivi.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene “X”, “B”.

Expected result: get(0) == “X”, get(1) == “B”.

- **public void testSetLastElement()**

Summary: Verifica che set(int index, Object element) funzioni correttamente sull’ultimo elemento della lista.

Test Method Design: Il test inserisce “A”, “B” e “C”, poi richiama set(2, “X”).

Test Description: L’ultimo elemento della lista si trova in posizione size() - 1. Il test controlla che il metodo set() possa modificare correttamente anche l’ultima posizione valida.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene “A”, “B”, “X”.

Expected result: get(0) == “A”; get(1) == “B”; get(2) == “X”.

- **public void testSetIndexGreaterThanSize()**

Summary: Verifica che set(int index, Object element) lanci IndexOutOfBoundsException quando l’indice è maggiore della dimensione della lista.

Test Method Design: Il test inserisce “A” nella lista e richiama set(2, “X”).

Test Description: Il metodo set() può operare solo su posizioni già esistenti. Un indice maggiore della dimensione della lista non individua alcun elemento da sostituire e deve quindi generare eccezione.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene “A”.

Expected result: set(2, “X”) deve lanciare IndexOutOfBoundsException.

- **public void testSetNullElement()**
 - Summary:** Verifica che `set(int index, Object element)` permetta di sostituire un elemento con null.
 - Test Method Design:** Il test inserisce “A” e “B”, poi richiama `set(1, null)`.
 - Test Description:** Il valore null deve essere accettato come elemento valido della lista. Il test controlla quindi che un elemento non nullo possa essere sostituito con null e che la dimensione rimanga invariata.
 - Pre-Condition:** La lista è appena stata creata e non contiene elementi.
 - Post-Condition:** La lista contiene “A”, null.
 - Expected result:** `get(1)` deve restituire null; `size() == 2`.

- **public void testSetReplacesNullElement()**
 - Summary:** Verifica che `set(int index, Object element)` possa sostituire un elemento null con un elemento non nullo.
 - Test Method Design:** Il test inserisce “A”, null e “C”, poi richiama `set(1, “B”)`.
 - Test Description:** Il test controlla il caso opposto rispetto all’inserimento di null: un valore null già presente nella lista deve poter essere sostituito da un altro oggetto. Il metodo deve inoltre restituire il vecchio elemento, cioè null.
 - Pre-Condition:** La lista è appena stata creata e non contiene elementi.
 - Post-Condition:** La lista contiene “A”, “B”, “C”.
 - Expected result:** Il valore restituito deve essere null; `get(1) == “B”`; `size() == 3`.

- **public void testSetNegative()**
 - Summary:** Verifica che `set(int index, Object element)` lanci `IndexOutOfBoundsException` quando l’indice è negativo.
 - Test Method Design:** Il test inserisce “A” nella lista e richiama `set(-1, “X”)`.
 - Test Description:** Un indice negativo non rappresenta una posizione valida della lista. Il metodo non deve quindi eseguire alcuna sostituzione e deve lanciare l’eccezione prevista.
 - Pre-Condition:** La lista è appena stata creata e non contiene elementi.
 - Post-Condition:** La lista contiene “A”.
 - Expected result:** `set(-1, “X”)` deve lanciare `IndexOutOfBoundsException`.

- **public void testSetIndexEqualToSize()**

Summary: Verifica che `set(int index, Object element)` lanci `IndexOutOfBoundsException` quando l'indice è uguale alla dimensione della lista.

Test Method Design: Il test inserisce "A" nella lista e richiama `set(1, "X")`.

Test Description: A differenza di `add(int index, Object element)`, il metodo `set()` non può usare l'indice `size()` perché deve sostituire un elemento già presente. In una lista di dimensione 1, l'indice 1 è quindi fuori dai limiti.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene A.

Expected result: `set(1, "X")` deve lanciare `IndexOutOfBoundsException`.

- **public void testSetOnEmptyList()**

Summary: Verifica che `set(int index, Object element)` lanci `IndexOutOfBoundsException` quando viene chiamato su una lista vuota.

Test Method Design: Il test richiama `set(0, "X")` su una lista appena creata.

Test Description: In una lista vuota non esistono elementi da sostituire. Anche l'indice 0 non è quindi valido per il metodo `set()`.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota

Expected result: `set(0, "X")` deve lanciare `IndexOutOfBoundsException`.

- **ListAdapter.add(int index, Object element)**

- **public void testAddAtBeginning()**

Summary: Verifica che add(int index, Object element) inserisca correttamente un elemento all'inizio della lista.

Test Method Design: Il test inserisce "B" e "C" nella lista, poi richiama add(0, "A").

Test Description: L'inserimento in posizione 0 deve aggiungere il nuovo elemento all'inizio della lista e spostare verso destra gli elementi già presenti.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "C".

Expected result: size() == 3; get(0) == "A"; get(1) == "B"; get(2) == "C".

- **public void testAddAtMiddle()**

Summary: Verifica che add(int index, Object element) inserisca correttamente un elemento in posizione intermedia.

Test Method Design: Il test inserisce "A" e "C", poi richiama add(1, "B").

Test Description: Il metodo deve inserire il nuovo elemento nell'indice indicato, mantenendo gli elementi precedenti prima della posizione di inserimento e spostando quelli successivi verso destra.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "C".

Expected result: size() == 3; get(0) == "A"; get(1) == "B"; get(2) == "C".

- **public void testAddAtEnd()**

Summary: Verifica che add(int index, Object element) consenta l'inserimento in coda quando l'indice è uguale a size().

Test Method Design: Il test inserisce "A" e "B", poi richiama add(2, "C").

Test Description: Per il metodo add(int index, Object element), l'indice uguale alla dimensione della lista è valido e rappresenta l'inserimento in coda. Il test controlla quindi questo caso limite valido.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "C".

Expected result: size() == 3; get(2) == "C".

- **public void testAddAtIndexZeroOnEmptyList()**

Summary: Verifica che add(int index, Object element) possa inserire un elemento in una lista vuota usando l'indice 0.

Test Method Design: Il test richiama add(0, "A") su una lista appena creata.

Test Description: In una lista vuota l'unico indice valido per l'inserimento è 0. Il test controlla che il metodo gestisca correttamente l'inserimento del primo elemento.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: size() == 1; get(0) == "A".

- **public void testAddNullAtIndex()**

Summary: Verifica che add(int index, Object element) consenta l'inserimento di un elemento null.

Test Method Design: Il test inserisce "A" e "B", poi richiama add(1, null).

Test Description: Il valore null deve essere considerato un elemento valido della lista. Il test controlla che possa essere inserito in una posizione specifica e che gli elementi successivi vengano spostati correttamente.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", null, "B".

Expected result: size() == 3; get(0) == "A"; get(1) == null; get(2) == "B".

- **public void testAddAtIndexShiftsElementsRight()**

(test già fatto implicitamente ma si è preferito farlo anche esplicitamente)

Summary: Verifica che add(int index, Object element) sposti verso destra gli elementi successivi all'indice di inserimento.

Test Method Design: Il test inserisce "A", "B" e "D", poi richiama add(2, "C").

Test Description: Quando un elemento viene inserito in una posizione già occupata, gli elementi che si trovano da quell'indice in poi devono essere spostati di una posizione verso destra. Il test controlla quindi che "D" non venga sovrascritto, ma semplicemente spostato.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "C", "D".

Expected result: get(0) == "A"; get(1) == "B"; get(2) == "C"; get(3) == "D".

- **public void testAddIndexGreaterThanOrEmptyList()**
Summary: Verifica che `add(int index, Object element)` lanci `IndexOutOfBoundsException` su lista vuota quando l'indice è maggiore di `size()`.
Test Method Design: Il test richiama `add(1, "A")` su una lista vuota.
Test Description: In una lista vuota `size()` è uguale a 0. L'indice 1 è quindi maggiore della dimensione della lista e non rappresenta una posizione valida di inserimento.
Pre-Condition: La lista è appena stata creata e non contiene elementi.
Post-Condition: La lista rimane vuota.
Expected result: `add(1, "A")` deve lanciare `IndexOutOfBoundsException`.

- **public void testAddNegativeIndex()**
Summary: Verifica che `add(int index, Object element)` lanci `IndexOutOfBoundsException` quando l'indice è negativo.
Test Method Design: Il test inserisce "A" nella lista e richiama `add(-1, "X")`.
Test Description: Gli indici validi per l'inserimento partono da 0. Un indice negativo non rappresenta quindi una posizione valida e deve produrre l'eccezione prevista.
Pre-Condition: La lista è appena stata creata e non contiene elementi.
Post-Condition: La lista contiene "A".
Expected result: `add(-1, "X")` deve lanciare `IndexOutOfBoundsException`.

- **public void testAddIndexGreaterThanOrSize()**
Summary: Verifica che `add(int index, Object element)` lanci `IndexOutOfBoundsException` quando l'indice è maggiore della dimensione della lista.
Test Method Design: Il test inserisce "A" nella lista e richiama `add(2, "X")`.
Test Description: Per il metodo `add()`, gli indici validi sono compresi tra 0 e `size()` inclusi. In una lista di dimensione 1, l'indice 2 è quindi fuori dai limiti.
Pre-Condition: La lista è appena stata creata e non contiene elementi.
Post-Condition: La lista contiene "A".
Expected result: `add(2, "X")` deve lanciare `IndexOutOfBoundsException`.

- **ListAdapter.remove(int index)**

- **public void testRemoveValidIndex()**

Summary: Verifica che remove(int index) rimuova correttamente l'elemento nella posizione indicata.

Test Method Design: Il test inserisce "A", "B" e "C", poi richiama remove(1).

Test Description: Il metodo remove(int index) deve eliminare l'elemento nella posizione specificata, restituirlo come valore di ritorno e aggiornare correttamente la struttura della lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "C".

Expected result: Il valore rimosso deve essere "B". size() == 2, get(0) == "A", get(1) == "C".

- **public void testRemoveFirstElement()**

Summary: Verifica che remove(int index) rimuova correttamente il primo elemento della lista.

Test Method Design: Il test inserisce "A", "B" e "C", poi richiama remove(0).

Test Description: La rimozione del primo elemento deve eliminare l'elemento in posizione 0 e spostare verso sinistra tutti gli elementi successivi.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "B", "C".

Expected result: Il valore rimosso deve essere "A", size() == 2, get(0) == "B". get(1) == "C".

- **public void testRemoveLastElement()**

Summary: Verifica che remove(int index) rimuova correttamente l'ultimo elemento della lista.

Test Method Design: Il test inserisce "A", "B" e "C", poi richiama remove(2).

Test Description: L'ultimo elemento si trova in posizione size() - 1. Il test controlla che il metodo remove() riesca a rimuovere correttamente anche l'ultimo indice valido.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B".

Expected result: Il valore rimosso deve essere "C", size() == 2, get(0) == "A", get(1) == "B".

- **public void testRemoveNullElementByIndex()**

Summary: Verifica che `remove(int index)` rimuova correttamente un elemento null.

Test Method Design: Il test inserisce "A", null e "C", poi richiama `remove(1)`.

Test Description: Il valore null deve essere trattato come un normale elemento della lista anche nelle operazioni indicizzate di rimozione. Il metodo deve restituire null e aggiornare correttamente la lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "C".

Expected result: Il valore rimosso deve essere null, `size() == 2`, `get(0) == "A"`, `get(1) == "C"`.

- **public void testRemoveShiftsElementsLeft()**

Summary: Verifica che `remove(int index)` sposti verso sinistra gli elementi successivi all'elemento rimosso.

Test Method Design: Il test inserisce "A", "B", "C" e "D", poi richiama `remove(1)`.

Test Description: Dopo la rimozione di un elemento centrale, tutti gli elementi successivi devono scalare di una posizione verso sinistra. Il test controlla quindi che la lista non lasci spazi vuoti e mantenga l'ordine relativo degli elementi rimasti.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "C", "D".

Expected result: `size() == 3`, `get(0) == "A"`, `get(1) == "C"`, `get(2) == "D"`.

- **public void testRemoveIndexGreaterThanSize()**

Summary: Verifica che `remove(int index)` lanci `IndexOutOfBoundsException` quando l'indice è maggiore della dimensione della lista.

Test Method Design: Il test inserisce "A" nella lista e richiama `remove(2)`.

Test Description: Il metodo `remove()` può eliminare solo elementi già presenti nella lista. Un indice maggiore della dimensione non individua alcun elemento valido e deve quindi generare eccezione.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: `remove(2)` deve lanciare `IndexOutOfBoundsException`.

- **public void testRemoveNegativeIndex()**

Summary: Verifica che `remove(int index)` lanci `IndexOutOfBoundsException` quando l'indice è negativo.

Test Method Design: Il test inserisce "A" nella lista e richiama `remove(-1)`.

Test Description: Un indice negativo non rappresenta una posizione valida della lista. Il metodo non deve quindi eseguire alcuna rimozione e deve lanciare l'eccezione prevista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: `remove(-1)` deve lanciare `IndexOutOfBoundsException`.

- **public void testRemoveIndexEqualToSize()**

Summary: Verifica che `remove(int index)` lanci `IndexOutOfBoundsException` quando l'indice è uguale alla dimensione della lista.

Test Method Design: Il test inserisce "A" nella lista e richiama `remove(1)`.

Test Description: In una lista di dimensione 1, l'unico indice valido è 0. L'indice 1, essendo uguale a `size()`, non corrisponde ad alcun elemento rimuovibile.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: `remove(1)` deve lanciare `IndexOutOfBoundsException`.

- **public void testRemoveOnEmptyList()**

Summary: Verifica che `remove(int index)` lanci `IndexOutOfBoundsException` quando viene chiamato su una lista vuota.

Test Method Design: Il test richiama `remove(0)` su una lista appena creata.

Test Description: In una lista vuota non esistono elementi da rimuovere. Anche l'indice 0 deve quindi essere considerato non valido.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: `remove(0)` deve lanciare `IndexOutOfBoundsException`.

ListAdapterSearchTest

Summary

Il test case ListAdapterSearchTest verifica il corretto funzionamento dei metodi di ricerca della classe ListAdapter.

L'obiettivo è controllare che i metodi indexOf(Object o) e lastIndexOf(Object o) individuino correttamente la posizione degli elementi all'interno della lista, distinguendo tra prima e ultima occorrenza.

I test considerano sia casi ordinari, sia casi limite, come lista vuota, elemento assente, elementi duplicati, valore null e confronto tra oggetti distinti ma uguali secondo il metodo equals(Object o).

Test Case Design

Ogni test utilizza una nuova istanza di ListAdapter, inizializzata prima dell'esecuzione tramite il metodo setUp().

I test sono organizzati in due gruppi principali: il primo riguarda indexOf(Object o), mentre il secondo riguarda lastIndexOf(Object o).

Per ogni metodo vengono verificati il valore intero restituito, il comportamento in caso di elemento presente o assente e la corretta gestione delle occorrenze multiple. I metodi di ricerca non devono modificare la lista, quindi lo stato della lista resta invariato dopo ogni operazione di questo tipo.

Test

- **ListAdapter.indexOf(Object o)**

- **public void testIndexOfOnEmptyList()**

Summary: Verifica che `indexOf(Object o)` restituisca -1 quando viene chiamato su una lista vuota.

Test Method Design: Il test richiama `indexOf("A")` su una lista appena creata.

Test Description: Poiché la lista non contiene alcun elemento, l'oggetto cercato non può essere presente. Il metodo deve quindi restituire -1, valore che indica l'assenza dell'elemento nella lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: `indexOf("A") == -1`.

- **public void testIndexOfExistingElement()**

Summary: Verifica che `indexOf(Object o)` restituisca l'indice corretto di un elemento presente nella lista.

Test Method Design: Il test inserisce nella lista gli elementi "A", "B" e "C", poi richiama `indexOf("B")`.

Test Description: Il metodo deve scorrere la lista e individuare la posizione dell'elemento cercato. In questo caso "B" si trova in posizione 1, quindi il valore restituito deve corrispondere a tale indice.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "C".

Expected result: `indexOf("B") == 1`.

- **public void testIndexOfNotExistingElement()**

Summary: Verifica che `indexOf(Object o)` restituisca -1 quando l'elemento cercato non è presente nella lista.

Test Method Design: Il test inserisce nella lista gli elementi "A" e "B", poi richiama `indexOf("C")`.

Test Description: La lista contiene elementi diversi da quello cercato. Il metodo deve quindi terminare la ricerca senza trovare corrispondenze e restituire -1, senza modificare il contenuto della lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B".

Expected result: `indexOf("C") == -1`.

- **public void testIndexOfFirstOccurrence()**

Summary: Verifica che `indexOf(Object o)` restituisca la prima occorrenza dell'elemento cercato.

Test Method Design: Il test inserisce nella lista "A", "B", "A" e "C", poi richiama `indexOf("A")`.

Test Description: Quando un elemento compare più volte nella lista, `indexOf(Object o)` deve restituire l'indice della prima occorrenza, cioè quella più vicina all'inizio della lista. In questo caso "A" è presente sia in posizione 0 sia in posizione 2, ma il metodo deve restituire 0.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "A" e "C".

Expected result: `indexOf("A") == 0`.

- **public void testIndexOfNullElement()**

Summary: Verifica che `indexOf(Object o)` gestisca correttamente la ricerca di un elemento null presente nella lista.

Test Method Design: Il test inserisce nella lista "A", null e "C", poi richiama `indexOf(null)`.

Test Description: Il valore null deve essere considerato un elemento valido della lista. Il metodo deve quindi riuscire a trovarlo e restituire l'indice in cui è contenuto, senza generare eccezioni.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", null e "C".

Expected result: `indexOf(null) == 1`.

- **public void testIndexOfNullNotPresent()**

Summary: Verifica che `indexOf(Object o)` restituisca -1 quando viene cercato null, ma la lista non contiene elementi null.

Test Method Design: Il test inserisce nella lista "A" e "B", poi richiama `indexOf(null)`.

Test Description: La lista contiene solo elementi non nulli. Il metodo deve quindi distinguere correttamente tra l'assenza di null e la presenza di altri elementi, restituendo -1.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B".

Expected result: `indexOf(null) == -1`.

- **public void testIndexOfFirstNullOccurrence()**

Summary: Verifica che `indexOf(Object o)` restituisca la prima occorrenza di null quando null è presente più volte.

Test Method Design: Il test inserisce nella lista "A", null, "B" e null, poi richiama `indexOf(null)`.

Test Description: Anche per il valore null vale la regola della prima occorrenza. Se null compare più volte nella lista, `indexOf(null)` deve restituire l'indice della prima posizione in cui compare.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", null, "B" e null.

Expected result: `indexOf(null) == 1`.

- **public void testIndexOfUsesEquals()**

Summary: Verifica che `indexOf(Object o)` usi `equals(Object o)` per confrontare gli oggetti.

Test Method Design: Il test crea due oggetti String distinti ma con lo stesso contenuto, inserisce il primo nella lista e cerca il secondo tramite `indexOf(Object o)`.

Test Description: Il metodo non deve confrontare gli oggetti soltanto per riferimento, ma deve basarsi sull'uguaglianza logica definita da `equals(Object o)`. In questo caso a1 e a2 sono due oggetti diversi, ma rappresentano entrambi la stringa "A".

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene un oggetto String con valore "A".

Expected result: `indexOf(a2) == 0`.

- **ListAdapter.lastIndexOf(Object o)**

- **public void testLastIndexOfOnEmptyList()**

Summary: Verifica che lastIndexOf(Object o) restituisca -1 quando viene chiamato su una lista vuota.

Test Method Design: Il test richiama lastIndexOf("A") su una lista appena creata.

Test Description: Poiché la lista non contiene elementi, l'oggetto cercato non può essere presente. Il metodo deve quindi restituire -1, indicando che non esiste alcuna occorrenza dell'elemento cercato.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: lastIndexOf("A") == -1.

- **public void testLastIndexOfExistingElement()**

Summary: Verifica che lastIndexOf(Object o) restituisca l'indice corretto di un elemento presente una sola volta nella lista.

Test Method Design: Il test inserisce nella lista "A", "B" e "C", poi richiama lastIndexOf("B").

Test Description: Quando l'elemento cercato è presente una sola volta, lastIndexOf(Object o) deve restituire lo stesso indice che restituirebbe indexOf(Object o). In questo caso "B" si trova in posizione 1.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B" e "C".

Expected result: lastIndexOf("B") == 1.

- **public void testLastIndexOfNotExistingElement()**

Summary: Verifica che lastIndexOf(Object o) restituisca -1 quando l'elemento cercato non è presente nella lista.

Test Method Design: Il test inserisce nella lista "A" e "B", poi richiama lastIndexOf("C").

Test Description: La lista contiene elementi diversi da quello cercato. Il metodo deve quindi indicare che non esiste alcuna occorrenza dell'elemento richiesto, restituendo -1.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B".

Expected result: lastIndexOf("C") == -1.

- **public void testLastIndexOfLastOccurrence()**

Summary: Verifica che `lastIndexOf(Object o)` restituisca l'ultima occorrenza dell'elemento cercato.

Test Method Design: Il test inserisce nella lista "A", "B", "A" e "C", poi richiama `lastIndexOf("A")`.

Test Description: Quando un elemento compare più volte nella lista, `lastIndexOf(Object o)` deve restituire l'indice dell'ultima occorrenza, cioè quella più vicina alla fine della lista. In questo caso "A" è presente sia in posizione 0 sia in posizione 2, ma il metodo deve restituire 2.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B", "A" e "C".

Expected result: `lastIndexOf("A") == 2`.

- **public void testLastIndexOfNullElement()**

Summary: Verifica che `lastIndexOf(Object o)` gestisca correttamente la ricerca di un elemento null presente nella lista.

Test Method Design: Il test inserisce nella lista "A", null e "C", poi richiama `lastIndexOf(null)`.

Test Description: Il valore null deve essere considerato un elemento valido della lista anche per la ricerca dell'ultima occorrenza. Se null è presente una sola volta, il metodo deve restituire l'indice in cui si trova.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", null e "C".

Expected result: `lastIndexOf(null) == 1`.

- **public void testLastIndexOfLastNullOccurrence()**

Summary: Verifica che `lastIndexOf(Object o)` restituisca l'ultima occorrenza di null quando null è presente più volte.

Test Method Design: Il test inserisce nella lista "A", null, "B" e null, poi richiama `lastIndexOf(null)`.

Test Description: Se il valore null compare più volte nella lista, `lastIndexOf(null)` deve restituire l'ultima posizione in cui compare. Il test controlla quindi che la ricerca venga effettuata considerando l'ultima occorrenza e non la prima.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", null, "B" e null.

Expected result: `lastIndexOf(null) == 3`.

- **public void testLastIndexOfNullNotPresent()**

Summary: Verifica che `lastIndexOf(Object o)` restituisca -1 quando viene cercato null, ma la lista non contiene elementi null.

Test Method Design: Il test inserisce nella lista "A" e "B", poi richiama `lastIndexOf(null)`.

Test Description: Poiché la lista non contiene null, il metodo deve restituire -1. Il test controlla quindi che null venga gestito correttamente anche quando non è presente.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B".

Expected result: `lastIndexOf(null) == -1`.

- **public void testLastIndexOfUsesEquals()**

Summary: Verifica che `lastIndexOf(Object o)` usi `equals(Object o)` per confrontare gli oggetti.

Test Method Design: Il test crea due oggetti String distinti ma con lo stesso contenuto, inserisce il primo nella lista e cerca il secondo tramite `lastIndexOf(Object o)`.

Test Description: Il metodo deve basarsi sull'uguaglianza logica degli oggetti e non solo sull'identità del riferimento. Anche se a1 e a2 sono due istanze diverse, risultano uguali secondo `equals(Object o)`, quindi la ricerca deve avere esito positivo.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene un oggetto String con valore "A".

Expected result: `lastIndexOf(a2) == 0`.

ListAdapterIteratorTest

Summary

Il test case ListAdapterIteratorTest verifica il corretto funzionamento dell'iteratore restituito dalla classe ListAdapter.

L'obiettivo è controllare che il metodo iterator() restituisca un oggetto valido e che i metodi hasNext(), next() e remove() dell'iteratore rispettino il comportamento previsto dalla documentazione. I test considerano sia casi ordinari, sia casi limite, come lista vuota, lista con un solo elemento, lista con più elementi, presenza di elementi null, chiamata a next() oltre la fine della lista e utilizzo non valido di remove().

Viene inoltre verificato che remove() elimini correttamente l'ultimo elemento restituito da next(), che possa essere chiamato una sola volta dopo ogni next() valido e che l'iteratore continui a funzionare correttamente dopo una rimozione.

Test Case Design

Ogni test utilizza una nuova istanza di ListAdapter, inizializzata prima dell'esecuzione tramite il metodo setUp().

I test sono organizzati in quattro gruppi principali: costruttore dell'iteratore, metodo iterator(), metodo hasNext(), metodo next() e metodo remove().

Per ogni metodo vengono verificati sia il valore restituito, quando presente, sia gli effetti prodotti sulla posizione dell'iteratore e sul contenuto della lista. Nei casi non validi viene controllato il corretto lancio delle eccezioni previste.

Test

- **Costruttore**

- **public void testIteratorConstructorNullList()**

Summary: Verifica che il costruttore di IteratorAdapter lanci NullPointerException quando riceve una lista nulla.

Test Method Design: Il test richiama direttamente new IteratorAdapter(null).

Test Description: Un iteratore deve essere associato a una lista valida. Il passaggio di una lista nulla rappresenta quindi un caso non valido e deve produrre l'eccezione prevista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota. Nessun iteratore viene creato.

Expected result: new IteratorAdapter(null) deve lanciare NullPointerException.

- **ListAdapter.iterator()**

- **public void testIteratorReturnsNotNull()**

Summary: Verifica che iterator() restituisca un iteratore non nullo.

Test Method Design: Il test richiama iterator() su una lista appena creata e controlla che il riferimento restituito non sia null.

Test Description: Il metodo iterator() deve sempre restituire un oggetto iteratore valido, anche quando la lista è vuota. Il test controlla quindi che la creazione dell'iteratore avvenga correttamente.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: list.iterator() deve restituire un oggetto diverso da null.

- **public void testIteratorOnEmptyListHasNoNext()**

Summary: Verifica che un iteratore creato su lista vuota non abbia elementi successivi.

Test Method Design: Il test crea un iteratore su una lista vuota e richiama hasNext().

Test Description: Se la lista non contiene elementi, l'iteratore non deve segnalare la presenza di un elemento successivo. Il test controlla quindi il comportamento dell'iteratore nel caso limite della lista vuota.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: hasNext() == false.

- **public void testIteratorOnNonEmptyListHasNext()**

Summary: Verifica che un iteratore creato su una lista non vuota abbia un elemento successivo.

Test Method Design: Il test inserisce un elemento nella lista, crea un iteratore e richiama hasNext().

Test Description: Quando la lista contiene almeno un elemento, l'iteratore appena creato deve essere posizionato prima del primo elemento e deve quindi indicare che esiste un elemento successivo da leggere.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: hasNext() == true.

- **IteratorAdapter.hasNext()**

- **public void testHasNextBeforeEnd()**

Summary: Verifica che hasNext() restituisca true finché non sono stati letti tutti gli elementi della lista.

Test Method Design: Il test inserisce due elementi nella lista, crea un iteratore, controlla hasNext(), richiama next() una volta e controlla nuovamente hasNext().

Test Description: L'iteratore deve segnalare la presenza di un elemento successivo sia all'inizio della scansione, sia dopo aver letto il primo elemento, finché rimane almeno un elemento ancora da visitare.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B". L'iteratore si trova tra "A" e "B".

Expected result: hasNext() deve restituire true prima della prima next() e anche dopo aver letto "A".

- **public void testHasNextAfterEnd()**

Summary: Verifica che hasNext() restituisca false dopo che tutti gli elementi sono stati letti.

Test Method Design: Il test inserisce due elementi nella lista, crea un iteratore, richiama next() due volte e poi controlla hasNext().

Test Description: Dopo aver visitato tutti gli elementi presenti nella lista, l'iteratore deve trovarsi alla fine della sequenza. In questa situazione non deve più segnalare elementi successivi.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B". L'iteratore si trova dopo "B".

Expected result: hasNext() == false.

- **public void testHasNextDoesNotMoveIterator()**

Summary: Verifica che hasNext() non modifichi la posizione dell'iteratore.

Test Method Design: Il test inserisce due elementi, richiama hasNext() e poi richiama next().

Test Description: Il metodo hasNext() deve limitarsi a controllare la presenza di un elemento successivo, senza avanzare l'iteratore. Per questo motivo, dopo la chiamata a hasNext(), la successiva chiamata a next() deve restituire ancora il primo elemento della lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B". L'iteratore si trova dopo "A" e prima di "B"

Expected result: hasNext() deve restituire true e la successiva next() deve restituire "A".

- **public void testHasNextWithNullNextElement()**

Summary: Verifica che hasNext() restituisca true anche quando il prossimo elemento è null.

Test Method Design: Il test inserisce null nella lista, crea un iteratore, controlla hasNext(), richiama next() e poi controlla nuovamente hasNext().

Test Description: Il valore null è un elemento valido della lista. La presenza di null come prossimo elemento non deve quindi essere interpretata come assenza di elementi. Il test controlla che hasNext() valuti la posizione dell'iteratore e non il valore dell'elemento successivo.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene un solo elemento null. L'iteratore si trova dopo null.

Expected result: hasNext() deve restituire true prima della lettura, next() deve restituire null e hasNext() deve restituire false dopo la lettura.

- **IteratorAdapter.next()**

- **public void testNextReturnsFirstElement()**

Summary: Verifica che next() restituisca il primo elemento della lista.

Test Method Design: Il test inserisce “A” e “B” nella lista, crea un iteratore e richiama next() una volta.

Test Description: Un iteratore appena creato deve essere posizionato prima del primo elemento. La prima chiamata a next() deve quindi restituire l'elemento in posizione 0.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene “A” e “B”. L'iteratore si trova tra “A” e “B”.

Expected result: next() == “A”.

- **public void testNextReturnsElementsInOrder()**

Summary: Verifica che next() restituisca gli elementi nell'ordine corretto.

Test Method Design: Il test inserisce “A”, “B” e “C” nella lista, crea un iteratore e richiama next() tre volte.

Test Description: L'iteratore deve attraversare la lista rispettando l'ordine degli elementi. Ogni chiamata a next() deve restituire l'elemento successivo rispetto alla posizione corrente dell'iteratore.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene “A”, “B” e “C”. L'iteratore si trova dopo “C”.

Expected result: Le chiamate a next() devono restituire, nell'ordine, “A”, “B” e “C”.

- **public void testNextReturnsNullElement()**

Summary: Verifica che next() restituisca correttamente un elemento null.

Test Method Design: Il test inserisce “A”, null e “C” nella lista, crea un iteratore e richiama next() tre volte.

Test Description: Il valore null deve essere trattato come un normale elemento della lista. Il metodo next() deve quindi restituirlo quando l'iteratore raggiunge la posizione in cui esso è contenuto, senza generare eccezioni.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene “A”, null e “C”. L'iteratore si trova dopo “C”.

Expected result: Le chiamate a next() devono restituire “A”, null e “C” nello stesso ordine.

- **public void testNextDoesNotModifyList()**

Summary: Verifica che next() restituisca l'elemento successivo senza modificare il contenuto della lista.

Test Method Design: Il test inserisce nella lista gli elementi "A", "B" e "C", crea un iteratore tramite iterator(), richiama next() una volta e poi controlla dimensione e contenuto della lista.

Test Description: Il metodo next() deve avanzare la posizione dell'iteratore e restituire l'elemento successivo, ma non deve effettuare modifiche strutturali sulla lista. In questo test, dopo la chiamata a next(), viene verificato che la lista mantenga la stessa dimensione e gli stessi elementi nello stesso ordine.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene ancora "A", "B" e "C"; l'iteratore si trova dopo "A" e prima di "B".

Expected result: next() == "A"; size() == 3; get(0) == "A"; get(1) == "B"; get(2) == "C".

- **public void testNextOnEmptyList()**

Summary: Verifica che next() lanci NoSuchElementException quando viene chiamato su una lista vuota.

Test Method Design: Il test crea un iteratore su una lista vuota e richiama next().

Test Description: Se la lista è vuota, non esiste alcun elemento successivo da restituire. Il metodo next() deve quindi segnalare l'errore lanciando NoSuchElementException.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: next() deve lanciare NoSuchElementException.

- **public void testNextAfterEnd()**

Summary: Verifica che next() lanci NoSuchElementException quando viene chiamato dopo la fine della lista.

Test Method Design: Il test inserisce un elemento, crea un iteratore, richiama next() una prima volta e poi richiama next() una seconda volta.

Test Description: Dopo aver letto l'unico elemento disponibile, l'iteratore si trova alla fine della lista. Una nuova chiamata a next() non può restituire alcun elemento e deve quindi lanciare NoSuchElementException.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene l'elemento "A".

Expected result: La seconda chiamata a next() deve lanciare NoSuchElementException.

- **IteratorAdapter.remove()**

- **public void testRemoveAfterNextRemovesElement()**

Summary: Verifica che remove() rimuova l'ultimo elemento restituito da next().

Test Method Design: Il test inserisce "A" e "B", richiama next(), salva l'elemento restituito e poi richiama remove().

Test Description: Dopo una chiamata valida a next(), remove() deve eliminare dalla lista proprio l'elemento appena restituito dall'iteratore. Il test controlla sia l'elemento letto, sia lo stato finale della lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "B". L'iteratore si trova prima di "B".

Expected result: next() == "A"; dopo remove(), size() == 1 e get(0) == "B".

- **public void testRemoveFirstElement()**

Summary: Verifica che remove() possa rimuovere il primo elemento della lista.

Test Method Design: Il test inserisce "A", "B" e "C", richiama next() una volta e poi remove().

Test Description: La prima chiamata a next() restituisce il primo elemento della lista. La successiva chiamata a remove() deve quindi rimuovere l'elemento in posizione iniziale e spostare correttamente gli elementi successivi.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "B", "C". L'iteratore si trova prima di "B".

Expected result: size() == 2; get(0) == "B"; get(1) == "C".

- **public void testRemoveMiddleElement()**

Summary: Verifica che remove() possa rimuovere un elemento centrale della lista.

Test Method Design: Il test inserisce "A", "B" e "C", richiama next() due volte e poi remove().

Test Description: Dopo due chiamate a next(), l'ultimo elemento restituito è "B". La chiamata a remove() deve quindi eliminare l'elemento centrale, lasciando nella lista gli elementi "A" e "C" nello stesso ordine relativo.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "C". L'iteratore si trova tra "A" e "C".

Expected result: size() == 2; get(0) == "A"; get(1) == "C".

- **public void testRemoveLastElement()**

Summary: Verifica che remove() possa rimuovere l'ultimo elemento della lista.

Test Method Design: Il test inserisce "A" e "B", richiama next() due volte e poi remove().

Test Description: Dopo aver letto entrambi gli elementi della lista, l'ultimo elemento restituito da next() è "B". La successiva chiamata a remove() deve rimuovere l'ultimo elemento, lasciando nella lista solo "A".

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A". L'iteratore si trova dopo "A".

Expected result: size() == 1; get(0) == "A".

- **public void testIteratorContinues()**

Summary: Verifica che l'iteratore continui correttamente la scansione dopo una rimozione.

Test Method Design: Il test inserisce "A", "B" e "C", legge "A", lo rimuove e poi continua a chiamare next().

Test Description: Dopo la rimozione dell'ultimo elemento restituito, l'iteratore deve aggiornare correttamente la propria posizione. La scansione deve proseguire dall'elemento successivo, senza saltare elementi e senza rileggere elementi già visitati.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "B" e "C". L'iteratore si trova dopo "C".

Expected result: Dopo la rimozione di "A", next() deve restituire "B", poi "C", e hasNext() deve restituire false.

- **public void testRemoveAfterEachNext()**

Summary: Verifica che remove() possa essere chiamato una volta dopo ogni next() valido.

Test Method Design: Il test inserisce "A", "B" e "C", poi alterna una chiamata a next() e una chiamata a remove() per ogni elemento.

Test Description: Il metodo remove() deve essere nuovamente abilitato dopo ogni chiamata valida a next(). Il test controlla quindi che sia possibile rimuovere progressivamente tutti gli elementi della lista tramite l'iteratore, purché ogni rimozione sia preceduta da una next() valida.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: La lista finale deve avere size() == 0 e isEmpty() == true.

- **public void testRemoveNullElement()**

Summary: Verifica che remove() funzioni correttamente anche quando l'elemento da rimuovere è null.

Test Method Design: Il test inserisce "A", null e "C", richiama next() fino a raggiungere null e poi richiama remove().

Test Description: Il valore null deve essere considerato un elemento valido anche per le operazioni dell'iteratore. Dopo che next() ha restituito null, remove() deve rimuovere quel valore dalla lista senza generare eccezioni e mantenendo l'ordine degli altri elementi.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "C". L'iteratore si trova tra "A" e "C".

Expected result: size() == 2; get(0) == "A"; get(1) == "C".

- **public void testRemoveBeforeNext()**

Summary: Verifica che remove() lanci RuntimeException se next() non è ancora stato chiamato.

Test Method Design: Il test inserisce "A" nella lista, crea un iteratore e richiama remove() senza prima chiamare next().

Test Description: Il metodo remove() dell'iteratore può rimuovere solo l'ultimo elemento restituito da next(). Se next() non è ancora stato chiamato, non esiste alcun elemento valido da rimuovere tramite l'iteratore.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A"

Expected result: remove() deve lanciare RuntimeException.

- **public void testRemoveTwiceAfterSameNext()**

Summary: Verifica che remove() lanci RuntimeException se viene chiamato due volte dopo la stessa chiamata a next().

Test Method Design: Il test inserisce "A" e "B", richiama next(), poi remove() due volte consecutive.

Test Description: Dopo una chiamata a next(), remove() può essere eseguito una sola volta. Una seconda chiamata a remove(), senza una nuova chiamata valida a next(), non ha un nuovo elemento da rimuovere e deve quindi produrre eccezione.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "B".

Expected result: La seconda chiamata a remove() deve lanciare RuntimeException.

ListAdapterListIteratorTest

Summary

Il test case ListAdapterListIteratorTest verifica il corretto funzionamento del list iterator associato alla classe ListAdapter.

L'obiettivo è controllare che il metodo listIterator() e il metodo listIterator(int index) restituiscano un iteratore valido e correttamente posizionato, e che i metodi specifici del ListIteratorAdapter gestiscano correttamente lo spostamento in avanti e all'indietro, il calcolo degli indici, la sostituzione, l'inserimento e la rimozione degli elementi.

In particolare, vengono testati i metodi:

- hasPrevious();
- previous();
- nextIndex();
- previousIndex();
- set(Object o);
- add(Object o);
- remove().

Poiché HListIterator estende HIterator, il list iterator deve rispettare anche il contratto dei metodi hasNext(), next() e remove(). Tuttavia, i metodi hasNext() e next() sono già stati verificati nel test case dedicato a IteratorAdapter, quindi in questo test case non vengono ritestati integralmente per evitare ridondanza (come da nota informativa).

Test Case Design

Ogni test utilizza una nuova istanza di ListAdapter, inizializzata prima dell'esecuzione tramite il metodo setUp().

I test sono organizzati per metodo testato. Prima vengono verificati i metodi della classe ListAdapter che creano il list iterator, cioè listIterator() e listIterator(int index). Successivamente vengono testati i metodi specifici dell'oggetto ListIteratorAdapter.

Le verifiche controllano il corretto valore restituito dai metodi, la posizione logica dell'iteratore, gli indici restituiti da nextIndex() e previousIndex(), la gestione degli elementi null, il corretto aggiornamento della lista dopo operazioni strutturali e il lancio delle eccezioni nei casi non validi.

Test

- **ListAdapter.listIterator()**

- **public void testListIteratorReturnsNotNull()**

Summary: Verifica che listIterator() restituisca un list iterator non nullo.

Test Method Design: Il test richiama list.listIterator() su una lista appena creata e controlla che il riferimento restituito non sia null.

Test Description: Il metodo listIterator() deve sempre restituire un oggetto iteratore valido, anche quando la lista è vuota. Il test controlla quindi che la creazione del list iterator avvenga correttamente.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: listIterator() deve restituire un oggetto diverso da null.

- **public void testListIteratorStartsAtZero()**

Summary: Verifica che listIterator() venga inizializzato all'inizio della lista.

Test Method Design: Il test inserisce alcuni elementi nella lista, crea un list iterator tramite listIterator() e controlla gli indici iniziali.

Test Description: Il metodo listIterator() senza parametri deve creare un iteratore posizionato prima del primo elemento della lista. Per questo motivo nextIndex() deve valere 0 e previousIndex() deve valere -1.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B". L'iteratore si trova prima di "A".

Expected result: nextIndex() == 0; previousIndex() == -1; una successiva chiamata a next() deve restituire "A".

- **ListAdapter.listIterator(index)**

- **public void testListIteratorStartsAtGivenIndex()**

Summary: Verifica che listIterator(int index) inizializzi l'iteratore nella posizione indicata.

Test Method Design: Il test inserisce più elementi nella lista e richiama listIterator(1), creando quindi un iteratore posizionato in una posizione intermedia.

Test Description: Il parametro index indica la posizione dell'elemento che verrebbe restituito da una successiva chiamata a next(). Se l'iteratore viene creato con indice 1, deve trovarsi tra l'elemento in posizione 0 e quello in posizione 1.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B" e "C". L'iteratore è tra "A" e "B".

Expected result: nextIndex() == 1; previousIndex() == 0; una successiva chiamata a next() deve restituire "B".

- **public void testListIteratorAtEnd()**

Summary: Verifica che listIterator(int index) possa creare un iteratore posizionato alla fine della lista.

Test Method Design: Il test inserisce alcuni elementi nella lista e richiama listIterator(size()), cioè con indice uguale alla dimensione della lista.

Test Description: L'indice uguale a size() è valido per un list iterator e rappresenta la posizione successiva all'ultimo elemento della lista. In questa posizione non deve esistere un elemento successivo, ma deve esistere un elemento precedente.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B". L'iteratore si trova dopo "B".

Expected result: hasNext() == false; hasPrevious() == true; nextIndex() == size(); previousIndex() == size() - 1.

- **public void testListIteratorNegativeIndex()**

Summary: Verifica che listIterator(int index) lanci IndexOutOfBoundsException quando l'indice è negativo.

Test Method Design: Il test richiama listIterator(-1).

Test Description: Un indice negativo non rappresenta una posizione valida della lista. Il metodo deve quindi impedire la creazione dell'iteratore e lanciare l'eccezione prevista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: listIterator(-1) deve lanciare IndexOutOfBoundsException.

- **public void testListIteratorIndexGreaterThanSize()**

Summary: Verifica che `listIterator(int index)` lanci `IndexOutOfBoundsException` quando l'indice è maggiore della dimensione della lista.

Test Method Design: Il test inserisce almeno un elemento nella lista e richiama `listIterator()` con un indice maggiore di `size()`.

Test Description: Per il metodo `listIterator(int index)`, gli indici validi sono compresi tra 0 e `size()` inclusi. Un indice maggiore di `size()` non individua una posizione valida dell'iteratore.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: `listIterator(2)` deve lanciare `IndexOutOfBoundsException`.

- **ListIteratorAdapter.hasPrevious()**

- **public void testHasPreviousOnEmptyList()**

Summary: Verifica che hasPrevious() restituisca false quando il list iterator è creato su una lista vuota.

Test Method Design: Il test crea un list iterator su una lista vuota e richiama hasPrevious().

Test Description: In una lista vuota non esiste alcun elemento precedente rispetto alla posizione corrente dell'iteratore. Il metodo deve quindi indicare correttamente l'assenza di elementi precedenti.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: hasPrevious() == false.

- **public void testHasPreviousAtBeginning()**

Summary: Verifica che hasPrevious() restituisca false quando l'iteratore è all'inizio della lista.

Test Method Design: Il test inserisce alcuni elementi nella lista, crea un list iterator all'inizio e richiama hasPrevious().

Test Description: Quando l'iteratore è posizionato prima del primo elemento, non esiste alcun elemento precedente. Il metodo deve quindi restituire false anche se la lista contiene elementi.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A". L'iteratore si trova prima di "A".

Expected result: hasPrevious() == false.

- **public void testHasPreviousAfterNext()**

Summary: Verifica che hasPrevious() restituisca true dopo una chiamata valida a next().

Test Method Design: Il test inserisce alcuni elementi nella lista, crea un list iterator, richiama next() e poi hasPrevious().

Test Description: Dopo una chiamata a next(), l'iteratore avanza di una posizione. A quel punto esiste un elemento precedente, cioè quello appena restituito da next().

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B". L'iteratore si trova tra "A" e "B".

Expected result: hasPrevious() == true.

- **public void testHasPreviousAtEnd()**

Summary: Verifica che hasPrevious() restituisca true quando l'iteratore è posizionato alla fine della lista.

Test Method Design: Il test inserisce alcuni elementi nella lista, crea un list iterator posizionato alla fine e richiama hasPrevious().

Test Description: Quando l'iteratore si trova dopo l'ultimo elemento della lista, non esiste un elemento successivo, ma esiste un elemento precedente. Il metodo deve quindi restituire true.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B". L'iteratore si trova dopo "B".

Expected result: hasPrevious() == true.

- **public void testHasPreviousDoesNotMoveIterator()**

Summary: Verifica che hasPrevious() non modifichi la posizione dell'iteratore.

Test Method Design: Il test posiziona l'iteratore in modo che esista un elemento precedente, richiama hasPrevious() e poi controlla che una successiva operazione sull'iteratore sia coerente con la posizione iniziale.

Test Description: Il metodo hasPrevious() deve limitarsi a controllare la presenza di un elemento precedente, senza spostare l'iteratore. Il test verifica quindi che la chiamata a hasPrevious() non consumi né modifichi alcun elemento.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B". L'iteratore si trova tra "A" e "B".

Expected result: it.previous() == "A".

- **ListIteratorAdapter.previous()**

- **public void testPreviousReturnsPreviousElement()**

Summary: Verifica che previous() restituisca l'elemento precedente rispetto alla posizione corrente dell'iteratore.

Test Method Design: Il test inserisce nella lista gli elementi "A" e "B", crea un list iterator in una posizione intermedia e richiama previous().

Test Description: Il metodo previous() deve restituire l'elemento che si trova immediatamente prima della posizione corrente dell'iteratore. Il test controlla quindi che, partendo da una posizione non iniziale, l'iteratore riesca a spostarsi correttamente all'indietro.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B"; l'iteratore si trova prima di "A".

Expected result: it.previous() == "A".

- **public void testPreviousReturnsElementsInReverseOrder()**

Summary: Verifica che previous() restituisca gli elementi della lista in ordine inverso.

Test Method Design: Il test inserisce nella lista gli elementi "A", "B" e "C", crea un list iterator posizionato alla fine della lista e richiama previous() tre volte.

Test Description: Partendo dalla fine della lista, ogni chiamata a previous() deve restituire l'elemento immediatamente precedente rispetto alla posizione corrente dell'iteratore. Il test controlla quindi che l'attraversamento all'indietro rispetti l'ordine inverso degli elementi presenti nella lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B" e "C"; l'iteratore si trova prima di "A".

Expected result: 3 chiamate successive di it.previous() devono restituire "C", "B", "A".

- **public void testPreviousReturnsNullElement()**

Summary: Verifica che previous() restituisca correttamente un elemento null.

Test Method Design: Il test inserisce nella lista gli elementi "A", null e "C", crea un list iterator posizionato dopo l'elemento null e richiama previous().

Test Description: Il valore null deve essere trattato come un normale elemento della lista. Il test controlla quindi che previous() restituisca null quando l'elemento precedente rispetto alla posizione corrente dell'iteratore è proprio null.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", null e "C"; l'iteratore si trova tra "A" e null.

Expected result: it.previous() == null.

- **public void testPreviousAtBeginning()**

Summary: Verifica che previous() lanci NoSuchElementException quando l'iteratore è posizionato all'inizio della lista.

Test Method Design: Il test inserisce almeno un elemento nella lista, crea un list iterator tramite listIterator(), quindi posizionato prima del primo elemento, e richiama previous().

Test Description: Quando il list iterator è all'inizio della lista, non esiste alcun elemento precedente da restituire. Il metodo previous() deve quindi segnalare questa situazione lanciando l'eccezione prevista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: it.previous() deve lanciare NoSuchElementException.

- **public void testPreviousOnEmptyList()**

Summary: Verifica che previous() lanci NoSuchElementException quando viene chiamato su una lista vuota.

Test Method Design: Il test crea un list iterator su una lista appena creata, senza inserire elementi, e richiama previous().

Test Description: In una lista vuota non esiste alcun elemento precedente rispetto alla posizione dell'iteratore. Il metodo previous() non può quindi restituire alcun valore e deve lanciare l'eccezione prevista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista rimane vuota.

Expected result: it.previous() deve lanciare NoSuchElementException.

- **ListIteratorAdapter.nextIndex() e ListIteratorAdapter.previousIndex()**

- **public void testInitialIndexes()**

Summary: Verifica che nextIndex() e previousIndex() restituiscano gli indici corretti quando il list iterator è appena stato creato all'inizio della lista.

Test Method Design: Il test inserisce nella lista gli elementi "A" e "B", crea un list iterator tramite list.listIterator() e controlla i valori restituiti da nextIndex() e previousIndex().

Test Description: Il metodo listIterator() crea un iteratore posizionato prima del primo elemento della lista. In questa posizione, il prossimo elemento che verrebbe restituito da next() è "A", che si trova in indice 0, mentre non esiste alcun elemento precedente.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B"; l'iteratore si trova prima di "A".

Expected result: it.nextIndex() == 0; it.previousIndex() == -1.

- **public void testIndexesAfterNext()**

Summary: Verifica che nextIndex() e previousIndex() vengano aggiornati correttamente dopo una chiamata a next().

Test Method Design: Il test inserisce nella lista gli elementi "A", "B" e "C", crea un list iterator tramite list.listIterator(), richiama next() una volta e poi controlla nextIndex() e previousIndex().

Test Description: All'inizio l'iteratore è posizionato prima di "A". Dopo la chiamata a next(), viene restituito "A" e l'iteratore si posiziona tra "A" e "B". In questa posizione, l'elemento precedente si trova in indice 0 e il prossimo elemento si trova in indice 1.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B" e "C"; l'iteratore si trova tra "A" e "B".

Expected result: it.nextIndex() == 1, it.previousIndex() == 0.

- **public void testIndexesAfterPrevious()**

Summary: Verifica che `nextIndex()` e `previousIndex()` vengano aggiornati correttamente dopo una chiamata a `previous()`.

Test Method Design: Il test inserisce nella lista gli elementi "A", "B" e "C", crea un list iterator in posizione 2, richiama `previous()` una volta e poi controlla `nextIndex()` e `previousIndex()`.

Test Description: L'iteratore creato con `listIterator(2)` si trova tra "B" e "C". Dopo la chiamata a `previous()`, viene restituito "B" e l'iteratore si sposta tra "A" e "B". In questa posizione, il prossimo elemento è "B" in indice 1, mentre l'elemento precedente è "A" in indice 0.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B" e "C"; l'iteratore si trova tra "A" e "B".

Expected result: `it.nextIndex() == 1`, `it.previousIndex() == 0`.

- **public void testIndexesAtEnd()**

Summary: Verifica che `nextIndex()` e `previousIndex()` restituiscano gli indici corretti quando il list iterator è posizionato alla fine della lista.

Test Method Design: Il test inserisce nella lista gli elementi "A", "B" e "C", crea un list iterator tramite `list.listIterator(list.size())` e controlla `nextIndex()` e `previousIndex()`.

Test Description: Quando l'iteratore è posizionato alla fine della lista, non esiste alcun elemento successivo. Tuttavia, il metodo `nextIndex()` deve restituire la dimensione della lista, mentre `previousIndex()` deve restituire l'indice dell'ultimo elemento.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B" e "C"; l'iteratore è posizionato dopo "C".

Expected result: `it.nextIndex() == 3`; `it.previousIndex() == 2`.

- **ListIteratorAdapter.set(Object o)**

- **public void testSetAfterNext()**

Summary: Verifica che set(Object o) sostituisca l'ultimo elemento restituito da next().

Test Method Design: Il test inserisce nella lista gli elementi "A" e "B", crea un list iterator tramite list.listIterator(), richiama next() e poi set("X").

Test Description: Dopo la chiamata a next(), l'ultimo elemento restituito dall'iteratore è "A". Il metodo set("X") deve quindi sostituire "A" con "X", senza modificare la dimensione della lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "X" e "B". L'iteratore si trova tra "X" e "B".

Expected result: it.next() == "A", get(0) == "X", get(1) == "B".

- **public void testSetAfterPrevious()**

Summary: Verifica che set(Object o) sostituisca l'ultimo elemento restituito da previous().

Test Method Design: Il test inserisce nella lista gli elementi "A" e "B", crea un list iterator alla fine della lista, richiama previous() e poi set("X").

Test Description: Il list iterator viene posizionato dopo "B". La chiamata a previous() restituisce "B"; quindi set("X") deve sostituire "B" con "X", senza modificare la dimensione della lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "X". L'iteratore si trova tra "A" e "X".

Expected result: it.previous() == "B", get(0) == "A", get(1) == "X".

- **public void testSetNullElement()**

Summary: Verifica che set(Object o) consenta di sostituire un elemento con null.

Test Method Design: Il test inserisce nella lista gli elementi "A", crea un list iterator, richiama next() e poi set(null).

Test Description: Dopo la chiamata a next(), l'ultimo elemento restituito è "A". Il metodo set(null) deve sostituire "A" con null, trattando null come un elemento valido della lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene null. L'iteratore si trova dopo null.

Expected result: get(0) == null.

- **public void testSetReplacesNullElement()**

Summary: Verifica che set(Object o) possa sostituire un elemento null con un elemento non nullo.

Test Method Design: Il test inserisce nella lista null, crea un list iterator, richiama next() e poi set("A").

Test Description: Dopo la chiamata a next(), l'ultimo elemento restituito è null. Il metodo set("A") deve sostituire il valore null con "A", senza modificare la dimensione della lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: get(0) == "A".

- **public void testSetBeforeNextOrPrevious()**

Summary: Verifica che set(Object o) lanci RuntimeException se non è ancora stato restituito alcun elemento.

Test Method Design: Il test inserisce nella lista l'elemento "A", crea un list iterator e richiama set("X") senza prima chiamare next() o previous().

Test Description: Il metodo set(Object o) può sostituire solo l'ultimo elemento restituito da next() o previous(). Poiché nessuno dei due metodi è stato ancora chiamato, non esiste alcun elemento valido da sostituire.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene ancora solo "A".

Expected result: set("X") deve lanciare RuntimeException;

- **public void testSetAfterAdd()**

Summary: Verifica che set(Object o) lanci RuntimeException se viene chiamato subito dopo add(Object o).

Test Method Design: Il test crea un list iterator, richiama add("A") e poi richiama set("X").

Test Description: Dopo una chiamata ad add(Object o), il metodo set(Object o) non può essere chiamato finché non viene eseguita una nuova chiamata valida a next() o previous(). Il test controlla quindi che questa regola venga rispettata.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A".

Expected result: set("X") deve lanciare RuntimeException.

- **public void testListIteratorSetAfterRemove()**

Summary: Verifica che `set(Object o)` lanci `RuntimeException` se viene chiamato subito dopo `remove()`.

Test Method Design: Il test inserisce nella lista gli elementi “A” e “B”, crea un list iterator, richiama `next()`, poi `remove()` e infine `set(“X”)`.

Test Description: Dopo `next()`, l’ultimo elemento restituito è “A”. La chiamata a `remove()` rimuove “A” dalla lista. Dopo una rimozione, `set(Object o)` non può essere chiamato finché non viene eseguita una nuova chiamata valida a `next()` o `previous()`.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene solo “B”.

Expected result: `set(“X”)` deve lanciare `RuntimeException`.

- **ListIteratorAdapter.add(Object o)**

- **public void testAddAtCurrentPosition()**

Summary: Verifica che add(Object o) inserisca un elemento nella posizione corrente dell'iteratore.

Test Method Design: Il test inserisce nella lista gli elementi "A" e "C", crea un list iterator in posizione 1 e richiama add("B").

Test Description: L'iteratore creato in posizione 1 si trova tra "A" e "C". Il metodo add(Object o) deve inserire il nuovo elemento nella posizione corrente, cioè prima dell'elemento che sarebbe restituito da una successiva chiamata a next().

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B" e "C"; l'iteratore si trova tra "B" e "C".

Expected result: list.get(0) == "A"; list.get(1) == "B"; list.get(2) == "C"; list.size() == 3.

- **public void testAddAtBeginning()**

Summary: Verifica che add(Object o) inserisca correttamente un elemento all'inizio della lista.

Test Method Design: Il test inserisce nella lista gli elementi "B", crea un list iterator tramite list.listIterator() e richiama add("A").

Test Description: Il list iterator creato con listIterator() è posizionato prima del primo elemento. In questa posizione, add(Object o) deve inserire il nuovo elemento all'inizio della lista, spostando gli elementi già presenti verso destra.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene, "A" e "B"; l'iteratore si trova tra "A" e "B".

Expected result: list.get(0) == "A", list.get(1) == "B", list.size() == 2.

- **public void testAddAtEnd()**

Summary: Verifica che add(Object o) inserisca correttamente un elemento alla fine della lista.

Test Method Design: Il test inserisce nella lista gli elementi "A", crea un list iterator tramite list.iterator(list.size()) e richiama add("B").

Test Description: Il list iterator viene creato alla fine della lista, cioè dopo "A". In questa posizione, add(Object o) deve inserire il nuovo elemento in coda.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "B"; l'iteratore si trova dopo "B".

Expected result: get(0) == "A", get(1) == "B", size() == 2.

- **public void testAddNullElement()**

Summary: Verifica che add(Object o) consenta l'inserimento di un elemento null.

Test Method Design: Il test crea un list iterator su una lista contenente "A" e "C" e richiama it.add(null).

Test Description: Il valore null deve essere trattato come un normale elemento della lista. Il test controlla quindi che il list iterator permetta di inserire null senza generare eccezioni.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene un "A", null, "C".

Expected result: size() == 3; get(0) == A, get(1) == null, get(2) == "C".

- **public void testPreviousAfterAddReturnsAddedElement()**

Summary: Verifica che previous() restituisca l'elemento appena inserito tramite add(Object o).

Test Method Design: Il test inserisce nella lista gli elementi "A" e "C", crea un list iterator in posizione 1, richiama add("B") e poi richiama previous().

Test Description: Dopo la chiamata ad add("B"), l'elemento "B" viene inserito nella posizione corrente dell'iteratore e l'iteratore si posiziona subito dopo l'elemento aggiunto. Per questo motivo, una successiva chiamata a previous() deve restituire proprio "B".

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B" e "C". L'iteratore si trova tra "A" e "B".

Expected result: it.previous() == "B".

- **public void testNextAfterAddReturnsFollowingElement()**

Summary: Verifica che next() restituisca l'elemento successivo corretto dopo una chiamata ad add(Object o).

Test Method Design: Il test inserisce nella lista gli elementi "A" e "C", crea un list iterator in posizione 1, richiama add("B") e poi richiama next().

Test Description: L'iteratore viene inizialmente posizionato tra "A" e "C". Dopo add("B"), l'elemento "B" viene inserito prima di "C" e l'iteratore si trova tra "B" e "C". Di conseguenza, la successiva chiamata a next() deve restituire "C", cioè l'elemento successivo alla posizione corrente.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A", "B" e "C"; l'iteratore si trova dopo "C".

Expected result: it.next() == "C".

- **ListIteratorAdapter.remove()**

- **public void testRemoveAfterPrevious()**

Summary: Verifica che remove() rimuova l'ultimo elemento restituito da previous().

Test Method Design: Il test inserisce nella lista gli elementi "A", "B" e "C", crea un list iterator in posizione 2, richiama previous() e poi remove().

Test Description: Il list iterator viene posizionato tra "B" e "C". La chiamata a previous() restituisce "B"; quindi remove() deve rimuovere proprio "B" dalla lista.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" e "C". L'iteratore si trova tra "A" e "C".

Expected result: it.previous() == "B"; dopo remove(), size() == 2; get(0) == "A", get(1) == "C".

- **public void testRemoveAfterNext()**

Summary: Verifica che remove() rimuova l'ultimo elemento restituito da next().

Test Method Design: Il test inserisce nella lista gli elementi "A" e "B", crea un list iterator, richiama next() e poi remove().

Test Description: Dopo la chiamata a next(), l'ultimo elemento restituito è "A". Il metodo remove() deve quindi rimuovere "A" dalla lista, lasciando "B" come primo elemento.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene solo "B". L'iteratore si trova prima di "B"

Expected result: it.next() == "A"; dopo remove(), size() == 1; get(0) == "B".

- **public void testRemoveBeforeNextOrPrevious()**

Summary: Verifica che remove() lanci RuntimeException se non è ancora stato restituito alcun elemento.

Test Method Design: Il test inserisce nella lista l'elemento "A", crea un list iterator e richiama remove() senza prima chiamare next() o previous().

Test Description: Il metodo remove() può rimuovere solo l'ultimo elemento restituito da next() o previous(). Poiché nessuno dei due metodi è stato ancora chiamato, non esiste alcun elemento valido da rimuovere.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene solo "A".

Expected result: remove() deve lanciare RuntimeException.

- **public void testRemoveAfterAdd()**

Summary: Verifica che remove() lanci RuntimeException se viene chiamato subito dopo add(Object o).

Test Method Design: Il test crea un list iterator, richiama add("A") e poi remove().

Test Description: Dopo una chiamata ad add(Object o), non esiste un ultimo elemento restituito da next() o previous() su cui applicare remove(). Per questo motivo la chiamata a remove() deve essere considerata non valida.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "A" .

Expected result: remove() deve lanciare RuntimeException.

- **public void testRemoveTwiceAfterSameNext()**

Summary: Verifica che remove() lanci RuntimeException se viene chiamato due volte dopo la stessa chiamata a next().

Test Method Design: Il test inserisce nella lista gli elementi "A" e "B", crea un list iterator, richiama next(), poi remove() e poi nuovamente remove().

Test Description: Dopo la chiamata a next(), l'ultimo elemento restituito è "A". Il primo remove() rimuove correttamente "A". Il secondo remove(), non essendo preceduto da una nuova chiamata valida a next() o previous(), deve generare eccezione.

Pre-Condition: La lista è appena stata creata e non contiene elementi.

Post-Condition: La lista contiene "B".

Expected result: il secondo remove() deve lanciare RuntimeException.

ListAdapterSubListTest

Summary

Il test case ListAdapterSubListTest verifica il corretto funzionamento della sottolista restituita dal metodo subList(int fromIndex, int toIndex) della classe ListAdapter.

L'obiettivo è controllare che la sottolista rappresenti correttamente una vista della porzione selezionata della lista principale, mantenendo dimensione, ordine degli elementi e indici relativi alla sottolista stessa.

I test verificano inoltre che le modifiche effettuate sulla sottolista si riflettano correttamente sulla lista madre, e viceversa nel caso di modifiche non strutturali. Vengono testate operazioni di accesso, sostituzione, inserimento, rimozione, svuotamento, ricerca, conversione in array, operazioni su collection, iteratori, confronto tramite equals(Object o), calcolo dell'hashCode() e gestione delle eccezioni sugli indici non validi.

Test Case Design

Ogni test utilizza una nuova istanza di ListAdapter, inizializzata prima dell'esecuzione tramite il metodo setUp().

A differenza di altri test case, in questo caso il metodo setUp() inizializza direttamente una lista popolata con gli elementi "A", "B", "C", "D" ed "E", perché quasi tutti i test sulla sottolista richiedono una lista madre già contenente più elementi.

I test sono organizzati in modo da verificare prima la creazione della sottolista e il suo contenuto, poi il comportamento dei principali metodi della lista applicati alla sottolista. Particolare attenzione viene data al fatto che la sottolista non sia una copia indipendente, ma una vista della lista principale: per questo vengono controllati gli effetti delle modifiche sia sulla sottolista sia sulla lista madre.

Le verifiche controllano quindi la dimensione della sottolista, il contenuto degli elementi, il corretto calcolo degli indici relativi, la propagazione delle modifiche, il comportamento degli iteratori e il corretto lancio di IndexOutOfBoundsException nei casi di indici non validi.

Test

- **ListAdapter.subList(int fromIndex, int toIndex)**

- **public void testSubListCorrectRange()**

Summary: Verifica che subList(int fromIndex, int toIndex) restituisca una vista contenente gli elementi compresi nel range indicato.

Test Method Design: Il test crea una sottolista tramite list.subList(1, 4) a partire da una lista contenente "A", "B", "C", "D" ed "E".

Test Description: Il metodo subList(1, 4) deve restituire una vista contenente gli elementi dalla posizione 1 inclusa alla posizione 4 esclusa della lista madre. La sottolista deve quindi contenere "B", "C" e "D", con indici relativi che partono da 0.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La lista principale rimane invariata; viene creata una sottolista contenente gli elementi del range richiesto.

Expected result: sub.size() == 3; sub.isEmpty() == false; sub.get(0) == "B"; sub.get(1) == "C"; sub.get(2) == "D".

- **public void testSubListEmpty()**

Summary: Verifica che sia possibile creare una sottolista vuota quando fromIndex e toIndex coincidono.

Test Method Design: Il test richiama list.subList(2, 2) su una lista contenente cinque elementi.

Test Description: Quando l'indice iniziale e l'indice finale sono uguali, il range selezionato non contiene elementi. Il metodo deve quindi restituire una sottolista vuota, senza generare eccezioni.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La lista principale rimane invariata; viene creata una sottolista vuota.

Expected result: sub.size() == 0; sub.isEmpty() == true.

- **SubList.set(int index, Object element)**

- **public void testSubListSetReflect()**

Summary: Verifica che `set(int index, Object element)` sulla sottolista modifichi anche la lista madre.

Test Method Design: Il test crea una sottolista con `list.subList(1, 4)`, corrispondente agli elementi "B", "C" e "D", poi richiama `sub.set(1, "X")`.

Test Description: La sottolista è una vista della lista principale, quindi una modifica non strutturale (non modifica la dimensione della lista) effettuata tramite `set()` sulla sottolista deve essere visibile anche nella lista madre. In questo caso l'elemento "C" della sottolista viene sostituito con "X".

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La sottolista contiene "B", "X" e "D"; la lista principale contiene "A", "B", "X", "D" ed "E".

Expected result: Il valore restituito da `sub.set(1, "X")` deve essere "C"; `sub.get(1) == "X"`; `list.get(2) == "X"`.

- **public void testParentListSetReflect1()**

Summary: Verifica che una modifica non strutturale effettuata sulla lista madre sia visibile anche nella sottolista.

Test Method Design: Il test crea una sottolista con `list.subList(1, 4)`, corrispondente agli elementi "B", "C" e "D", poi modifica la lista principale tramite `list.set(2, "X")`.

Test Description: Poiché la sottolista è una vista della lista principale, la sostituzione di un elemento della lista madre appartenente al range della sottolista deve riflettersi anche nella sottolista stessa.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La lista principale contiene "A", "B", "X", "D" ed "E"; la sottolista contiene "B", "X" e "D".

Expected result: Dopo `list.set(2, "X")`, `sub.get(1) == "X"`.

- **SubList.add(int index, Object element)**

- **public void testSubListAddByIndexReflect()**

Summary: Verifica che add(int index, Object element) sulla sottolista inserisca l'elemento nella posizione corretta e modifichi anche la lista madre.

Test Method Design: Il test crea una sottolista con list.subList(1, 4), corrispondente agli elementi "B", "C" e "D", poi richiama sub.add(1, "X").

Test Description: L'inserimento indicizzato sulla sottolista deve usare gli indici relativi della sottolista. L'elemento "X" deve quindi essere inserito in posizione 1 della sottolista, tra "B" e "C", e nella posizione corrispondente della lista madre.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La sottolista contiene "B", "X", "C" e "D"; la lista principale contiene "A", "B", "X", "C", "D" ed "E".

Expected result: sub.size() == 4; sub.get(0) == "B"; sub.get(1) == "X"; sub.get(2) == "C"; sub.get(3) == "D"; list.size() == 6; list.get(0) == "A"; list.get(1) == "B"; list.get(2) == "X"; list.get(3) == "C"; list.get(4) == "D"; list.get(5) == "E".

- **SubList.add(Object o)**

- **public void testSubListAddObjectReflect()**

Summary: Verifica che add(Object o) sulla sottolista aggiunga l'elemento alla fine della sottolista e modifichi anche la lista madre.

Test Method Design: Il test crea una sottolista con list.subList(1, 4), corrispondente agli elementi "B", "C" e "D", poi richiama sub.add("X").

Test Description: Il metodo add(Object o) sulla sottolista deve inserire il nuovo elemento alla fine della porzione rappresentata dalla sottolista. L'elemento "X" deve quindi essere aggiunto dopo "D" e prima di "E" nella lista madre.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La sottolista contiene "B", "C", "D" e "X"; la lista principale contiene "A", "B", "C", "D", "X" ed "E".

Expected result: sub.add("X") == true; sub.size() == 4; sub.get(3) == "X"; list.size() == 6; list.get(4) == "X"; list.get(5) == "E".

- **SubList.remove(int index)**

- **public void testSubListRemoveByIndexReflect()**

Summary: Verifica che remove(int index) sulla sottolista rimuova l'elemento corretto e modifichi anche la lista madre.

Test Method Design: Il test crea una sottolista con list.subList(1, 4), corrispondente agli elementi "B", "C" e "D", poi richiama sub.remove(1).

Test Description: La rimozione deve avvenire usando gli indici relativi della sottolista. L'indice 1 della sottolista corrisponde all'elemento "C", che deve quindi essere rimosso sia dalla sottolista sia dalla lista madre.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La sottolista contiene "B" e "D"; la lista principale contiene "A", "B", "D" ed "E".

Expected result: Il valore restituito da sub.remove(1) deve essere "C"; sub.size() == 2; sub.get(0) == "B"; sub.get(1) == "D"; list.size() == 4; list.get(0) == "A"; list.get(1) == "B"; list.get(2) == "D"; list.get(3) == "E".

- **SubList.clear()**

- **public void testSubListClearOnlySubRange()**

Summary: Verifica che clear() sulla sottolista rimuova tutti e solo gli elementi appartenenti al suo range.

Test Method Design: Il test crea una sottolista con list.subList(1, 4), corrispondente agli elementi "B", "C" e "D", poi richiama sub.clear().

Test Description: Il metodo clear() applicato alla sottolista deve eliminare dalla lista madre solo gli elementi inclusi nel range della sottolista, lasciando invariati gli elementi esterni al range.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La sottolista è vuota; la lista principale contiene solo gli elementi esterni al range originario della sottolista.

Expected result: sub.size() == 0; list.size() == 2; list.get(0) == "A"; list.get(1) == "E".

- **SubList.contains(Object o), indexOf(Object o), lastIndexOf(Object o)**

- **public void testSubListSearchOnlySubRange()**

Summary: Verifica che i metodi di ricerca della sottolista operino solo sugli elementi contenuti nel range della sottolista.

Test Method Design: Il test svuota la lista principale, inserisce gli elementi "A", "B", "A", "C", "A" e "D", poi crea una sottolista con `list.subList(1, 5)`.

Test Description: La sottolista contiene "B", "A", "C" e "A". Il test controlla che `contains(Object o)`, `indexOf(Object o)` e `lastIndexOf(Object o)` considerino solo questi elementi, ignorando quelli esterni al range della sottolista. Inoltre, gli indici restituiti devono essere relativi alla sottolista e non alla lista madre.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La lista principale e la sottolista rimangono invariate.

Expected result: `sub.contains("B") == true`; `sub.contains("A") == true`; `sub.contains("C") == true`; `sub.contains("D") == false`; `sub.indexOf("A") == 1`; `sub.lastIndexOf("A") == 3`; `sub.indexOf("B") == 0`; `sub.indexOf("C") == 2`; `sub.indexOf("D") == -1`; `sub.lastIndexOf("D") == -1`.

- **SubList.containsAll(HCollection c), addAll(HCollection c), removeAll(HCollection c), retainAll(HCollection c)**

- **public void testSubListCollectionOperations()**

Summary: Verifica il comportamento delle operazioni bulk applicate a una sottolista.

Test Method Design: Il test crea una sottolista con `list.subList(1, 4)`, corrispondente agli elementi "B", "C" e "D", e utilizza diverse istanze di `ListAdapter` come collection di supporto per verificare `containsAll()`, `addAll()`, `removeAll()` e `retainAll()`.

Test Description: I metodi bulk sono già testati in modo completo sulla classe `ListAdapter`. In questo test viene verificato che, quando tali metodi sono applicati a una sottolista, essi operino solo sul range della sottolista e che le modifiche strutturali vengano riflesse correttamente anche nella lista madre.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: Dopo le operazioni bulk, la sottolista contiene solo "D"; la lista principale contiene "A", "D" ed "E".

Expected result: `sub.containsAll(contained) == true;`
`sub.containsAll(notContained) == false;` dopo `sub.addAll(toAdd)`, `sub.size() == 5`, `sub.get(0) == "B"`, `sub.get(1) == "C"`, `sub.get(2) == "D"`, `sub.get(3) == "X"`, `sub.get(4) == "Y"`; nella lista madre `list.get(4) == "X"`, `list.get(5) == "Y"`, `list.get(6) == "E"`; dopo `sub.removeAll(toRemove)`, `sub.size() == 3`, `sub.get(0) == "C"`, `sub.get(1) == "D"`, `sub.get(2) == "X"`; dopo `sub.retainAll(toRetain)`, `sub.size() == 1`, `sub.get(0) == "D"`, `list.size() == 3`, `list.get(0) == "A"`, `list.get(1) == "D"`, `list.get(2) == "E"`.

- **SubList.subList(int fromIndex, int toIndex)**

- **public void testSubListOfSubList()**

Summary: Verifica che una sottolista creata a partire da un'altra sottolista gestisca correttamente gli offset.

Test Method Design: Il test crea prima una sottolista con `list.subList(1, 5)`, corrispondente agli elementi "B", "C", "D" ed "E", poi crea una seconda sottolista tramite `sub.subList(1, 3)`.

Test Description: Quando una sottolista viene creata a partire da un'altra sottolista, gli indici devono essere calcolati correttamente rispetto all'offset della lista madre. La seconda sottolista deve contenere "C" e "D". Inoltre, una modifica effettuata sulla seconda sottolista deve riflettersi sia sulla prima sottolista sia sulla lista principale.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La seconda sottolista contiene "X" e "D"; la prima sottolista e la lista madre mostrano la stessa modifica.

Expected result: `subSub.size() == 2`; `subSub.get(0) == "C"`; `subSub.get(1) == "D"`; dopo `subSub.set(0, "X")`, `subSub.get(0) == "X"`; `sub.get(1) == "X"`; `list.get(2) == "X"`.

- **SubList.toArray()**

- **public void testSubListToArray()**

Summary: Verifica che `toArray()` sulla sottolista restituisca un array contenente solo gli elementi della sottolista.

Test Method Design: Il test crea una sottolista con `list.subList(1, 4)`, corrispondente agli elementi "B", "C" e "D", e richiama `sub.toArray()`.

Test Description: Il metodo `toArray()` deve convertire in array solo la porzione rappresentata dalla sottolista, mantenendo l'ordine degli elementi. Non devono essere inclusi elementi della lista madre esterni al range della sottolista.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La lista principale e la sottolista rimangono invariate.

Expected result: L'array restituito deve avere lunghezza 3; `array[0] == "B"`; `array[1] == "C"`; `array[2] == "D"`.

- **SubList.toArray(Object[] a)**

- **public void testSubListToArrayObject()**

Summary: Verifica che `toArray(Object[] a)` sulla sottolista copi correttamente gli elementi in un array più grande della sottolista.

Test Method Design: Il test crea una sottolista con `list.subList(1, 4)`, corrispondente agli elementi "B", "C" e "D", e passa al metodo un array di lunghezza 5 già contenente alcuni valori.

Test Description: Quando l'array passato è più grande della sottolista, il metodo deve riutilizzare lo stesso array, copiare gli elementi della sottolista nelle prime posizioni e inserire null subito dopo l'ultimo elemento copiato. Gli elementi successivi dell'array devono rimanere invariati.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La lista principale e la sottolista rimangono invariate; l'array passato viene riutilizzato e aggiornato nelle prime posizioni.

Expected result: result deve essere lo stesso oggetto di input; `result[0] == "B"`; `result[1] == "C"`; `result[2] == "D"`; `result[3] == null`; `result[4] == "K"`.

- **SubList.iterator() e listIterator(int index)**

- **public void testSubListIterators()**

Summary: Verifica che gli iteratori della sottolista attraversino solo gli elementi appartenenti al range della sottolista.

Test Method Design: Il test crea una sottolista con `list.subList(1, 4)`, corrispondente agli elementi "B", "C" e "D", poi utilizza sia un `HIterator` sia un `HListIterator` sulla sottolista.

Test Description: Gli iteratori sono già testati in modo completo sui test case dedicati a `IteratorAdapter` e `ListIteratorAdapter`. In questo test viene verificato che, quando gli iteratori sono creati su una sottolista, essi operino soltanto sugli elementi del range della sottolista e restituiscano indici relativi alla sottolista stessa.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La lista principale e la sottolista rimangono invariate; gli iteratori hanno attraversato solo gli elementi della sottolista.

Expected result: L'iteratore semplice deve avere `hasNext() == true`, poi deve restituire "B", "C" e "D" con chiamate successive a `next()`, infine `hasNext() == false`; il list iterator creato con `sub.listIterator(1)` deve avere `hasNext() == true`, `hasPrevious() == true`, `nextIndex() == 1`, `previousIndex() == 0`; deve restituire "C" con `next()`, poi "C" e "B" con due chiamate successive a `previous()`, infine `hasPrevious() == false`.

- **SubList.equals(Object o) e hashCode()**

- **public void testSubListEqualsAndHashCode()**

Summary: Verifica che equals(Object o) e hashCode() della sottolista considerino solo gli elementi del range della sottolista.

Test Method Design: Il test crea una sottolista con list.subList(1, 4) e una seconda lista contenente gli stessi elementi della sottolista, cioè "B", "C" e "D".

Test Description: La sottolista deve comportarsi come una lista rispetto al confronto tramite equals(Object o) e al calcolo di hashCode(). Il confronto deve basarsi solo sugli elementi della sottolista e sul loro ordine, senza considerare gli elementi esterni presenti nella lista madre.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La lista principale, la sottolista e la lista other rimangono invariate.

Expected result: sub.equals(other) == true; other.equals(sub) == true; sub.hashCode() == other.hashCode().

- **Eccezioni su creazione subList e su metodi indicizzati**

- **public void testSubListException()**

Summary: Verifica che `subList(int fromIndex, int toIndex)` lanci `IndexOutOfBoundsException` quando gli indici di creazione non sono validi.

Test Method Design: Il test richiama `subList()` con tre combinazioni di indici non valide: indice iniziale negativo, indice finale maggiore della dimensione della lista e indice iniziale maggiore dell'indice finale.

Test Description: Il metodo `subList()` deve accettare solo intervalli validi, cioè con `fromIndex >= 0`, `toIndex <= size()` e `fromIndex <= toIndex`. Qualsiasi violazione di queste condizioni deve produrre `IndexOutOfBoundsException`.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La lista principale rimane invariata.

Expected result: `list.subList(-1, 3)` deve lanciare `IndexOutOfBoundsException`; `list.subList(1, 6)` deve lanciare `IndexOutOfBoundsException`; `list.subList(4, 2)` deve lanciare `IndexOutOfBoundsException`.

- **public void testSubListIndexedException()**

Summary: Verifica che i metodi indicizzati della sottolista lancino `IndexOutOfBoundsException` quando ricevono indici non validi.

Test Method Design: Il test crea una sottolista con `list.subList(1, 4)`, avente dimensione 3, e richiama diversi metodi indicizzati con valori fuori dai limiti.

Test Description: I metodi `get(int index)`, `set(int index, Object element)`, `add(int index, Object element)` e `remove(int index)` devono controllare gli indici rispetto alla dimensione della sottolista, non rispetto alla dimensione della lista madre. Gli indici negativi, gli indici uguali alla dimensione per operazioni di accesso/sostituzione/rimozione e gli indici maggiori della dimensione per inserimento devono generare eccezione.

Pre-Condition: La lista principale contiene "A", "B", "C", "D" ed "E".

Post-Condition: La lista principale e la sottolista rimangono invariate.

Expected result: `sub.get(-1)` deve lanciare `IndexOutOfBoundsException`; `sub.get(3)` deve lanciare `IndexOutOfBoundsException`; `sub.set(3, "X")` deve lanciare `IndexOutOfBoundsException`; `sub.add(4, "X")` deve lanciare `IndexOutOfBoundsException`; `sub.remove(3)` deve lanciare `IndexOutOfBoundsException`.